



**COMSATS University Islamabad,
Sahiwal Campus.**

Dr.Genie

**A Project Presented To
COMSATS University Islamabad, Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2022-2026)

By

**Laiba Waseem
Ayesha Fatima
Muhammad Farzam Imdad**

**CIIT/SP22-BCS-175/SWL
CIIT/SP22-BCS-188/SWL
CIIT/SP22-BCS-191/SWL**



**COMSATS University Islamabad,
Sahiwal Campus.**

Dr.Genie

By

**Laiba Waseem
Ayesha Fatima
Muhammad Farzam Imdad**

**CIIT/SP22-BCS-175/SWL
CIIT/SP22-BCS-188/SWL
CIIT/SP22-BCS-191/SWL**

Supervisor
Ms. Tahreem Saeed

Bachelor of Science in Computer Science (2022-2026)

**The candidate confirms that the work submitted is their own and appropriate credit
has been given where reference has been made to the work of other**

Declaration

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Laiba Waseem

Ayesha Fatima

Muhammad Farzam Imdad

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “**Dr. Genie**” was developed by **Laiba Waseem** (CIIT/SP22-BCS-175/SWL) , **Ayesha Fatima** (CIIT/SP22-BCS-188/SWL) and **Farzam Imdad** (CIIT/SP22-BCS-191/SWL) under the supervision of “**Ms. Tahreem Saeed**” and co supervisor “**Dr. Zafar Iqbal Roy**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Ms. Tahreem Saeed

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Javeed Rashid

Assistant Professor

Department of Computer Science

University Of Okara

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Dr. Genie is an AI-powered mobile application designed to revolutionize diabetes management by integrating machine learning-based disease prediction, image-based complication detection, personalized nutrition recommendations, real-time health tracking, and emergency assistance. The system incorporates multiple machine learning models including **Diabetes Risk Prediction**, **Retinopathy Detection**, and **Neuropathy (Diabetic Foot) Detection**, supported by Firebase backend services and a user-friendly Flutter frontend.

The project successfully delivers:

- ML-driven diabetes risk prediction
- Retinopathy and neuropathy image analysis
- Personalized diet and meal planning
- AI chatbot for real-time guidance
- Blood glucose trend tracking
- Comprehensive health records management
- Emergency handling module

This unified system eliminates the fragmentation found in existing healthcare apps and provides a complete digital healthcare assistant for diabetic patients

.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Ms. Tahreem Saeed” and our Co-Supervisor “Dr. Zafar Iqbal Roy”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Laiba Waseem

Ayesha Fatima

Muhammad Farzam Imdad

Abbreviations

Abbreviation	Meaning
DR	Diabetic Retinopathy
API	Application Programming Interface
UI	User Interface
UX	User Experience
TFLite	TensorFlow Lite
BMI	Body Mass Index
SDK	Software Development Kit
REST	Representational State Transfer
JSON	JavaScript Object Notation

Table of Contents

1	Introduction.....	1
1.1	Brief	1
1.2	Relevance to Course Modules	2
1.2.1	Web Technologies (CSC312)	2
1.2.2	Human-Computer Interaction (CSC356).....	3
1.2.3	Operating Systems (CSC322)	3
1.2.4	Database Systems I (CSC371).....	3
1.3	Project Background.....	4
1.4	Literature Review.....	4
1.4.1	Machine Learning for Diabetes Prediction	4
1.4.2	Deep Learning for Diabetic Retinopathy	5
1.4.3	Diabetic Foot Ulcer Detection	5
1.4.4	Mobile Health App Effectiveness	5
1.4.5	Analysis from Literature Review	5
1.5	Methodology and Software Lifecycle for This Project.....	5
1.5.1	Key Phases	6
1.5.2	Rationale behind Selected Methodology.....	6
2	Problem Definition	8
2.1.1	Problem Statement	9
2.1.2	Machine Learning–Based Diabetes Prediction	11
2.1.3	Deep Learning Image-Based Diagnosis.....	12
2.1.4	Personalized Diet & Meal Recommendations	12
2.1.5	AI Chatbot for Guidance.....	13
2.1.6	Blood Glucose Monitoring & Trends	13
2.1.7	Health Record Management	13

2.1.8	Emergency Handling Module	13
2.1.9	Cloud-Based Data Synchronization	14
2.2	Deliverables and Development Requirements	14
2.2.1	Software Deliverables	14
2.2.2	Documentation Deliverables	14
2.2.3	Technical Requirements	14
2.3	Current System	15
2.3.1	Existing System Weaknesses	15
2.3.2	Dr. Genie Improvements	15
3	Requirement Analysis	16
3.1	Use Case Description	17
3.2	Functional Requirements	23
3.2.1	Authentication Module	23
3.2.2	Diabetes Prediction Module	23
3.2.3	Retinopathy Detection Module	23
3.2.4	Neuropathy/Foot Ulcer Detection Module	23
3.2.5	Diet Recommendation Module	23
3.2.6	Chatbot Module	23
3.2.7	Emergency Help	23
3.2.8	Health Record Management	24
3.2.9	Glucose Monitoring	24
3.3	Non-Functional Requirements	24
3.3.1	Performance Requirements	24
3.3.2	Security Requirements	24
3.3.3	Usability Requirements	24
3.3.4	Reliability Requirements	24
3.3.5	Maintainability Requirements	24

3.3.6	Scalability Requirements	25
3.3.7	Portability Requirements	25
4	Design and Architecture.....	26
4.1	System Architecture	26
4.2	Data Representation [Diagram + Description]	27
4.3	Process Flow/Representation	27
4.4	Sequence Diagram	31
4.4.1	Sequence Diagram of Authentication	31
4.4.2	Sequence Diagram of Diabetes Prediction	32
4.4.3	Sequence Diagram of Retinopathy Detection.....	33
4.4.4	Sequence Diagram of Neuropathy Detection	34
4.4.5	Sequence Diagram of Meal Planner	35
4.4.6	Sequence Diagram of AI Chatbot.....	35
4.5	Class Diagram.....	36
5	Implementation	38
5.1	Development Environment & Tools.....	38
5.1.1	Frontend Development (Flutter)	38
5.1.2	Backend Development (Flask API)	38
5.1.3	Machine Learning & Deep Learning Models (Python)	39
5.1.4	Firebase Integration	39
5.1.5	On-Device Inference (TensorFlow Lite)	39
5.2	Core Algorithms.....	39
5.2.1	Diabetes Prediction Algorithm	39
5.2.2	Retinopathy Detection Algorithm (Convolutional Neural Network)	42
5.2.3	Neuropathy (Diabetic Foot Ulcer) Detection Algorithm (MobileNetV2 + TFLite).....	44
5.2.4	Diet Recommendation Algorithm (Hybrid ML + Rule-Based Engine)	45
5.3	External APIs	46

5.3.1	Prediction API (Flask)	46
5.3.2	Retinopathy Detection API	47
5.3.3	Chatbot API	47
5.4	User Interface	47
6	Testing and Evaluation	62
6.1	Testing Strategy	62
6.2	Manual Testing	62
6.3	System testing	63
6.4	Unit Testing	64
6.4.1	Unit Testing Flutter	64
6.4.2	ML API Unit Testing	65
6.5	Functional Testing	65
6.6	Machine Learning Model Evaluation	66
6.6.1	Diabetes Prediction Model (ML)	66
6.6.2	Retinopathy Detection Model (CNN)	67
6.6.3	Neuropathy Foot Ulcer Detection (TFLite)	67
6.7	Integration Testing	67
6.8	Performance Testing:	68
6.9	UI/UX Testing	68
6.10	Bug Fixes and Improvements	68
7	Conclusion and Future Work	69
7.1	Conclusion	69
7.2	Future Work	71
7.2.1	Real-Time Continuous Glucose Monitoring (CGM)	71
7.2.2	Advanced Meal Recommendation using AI	71
7.2.3	Expand Image Datasets for Better Accuracy	71
7.2.4	Offline-First Version of the App	72

7.3	Final Thoughts	72
	References	73

List of Figures

Figure 3.1. Use Case Diagram	16
Figure 4.1: Architecture Diagram	25
Figure 4.2: Flow Diagram.....	27
Figure 4.3: Sequence Diagram of Authentication.....	29
Figure 4.3: Sequence Diagram of Diabetes Prediction	30
Figure 4.3: Sequence Diagram of Retinopathy Detection	31
Figure 4.3: Sequence Diagram of Neuropathy Detection	31
Figure 4.3: Sequence Diagram of Meal Planner	32
Figure 4.3: Sequence Diagram of AI Chatbot	33
Figure 4.4: Class Diagram	34
Figure 5.1:DashBoard Screen	45
Figure 5.2: Authentication Screens.....	46
Figure 5.3:Diabetes Prediction Screen.....	47
Figure 5.4:Retinopathy Detection Screen	48
Figure 5.5:Neuropathy Detection Screen.....	49
Figure 5.6:Chatbot Screen	50
Figure 5.7:Meal Planner Screen.....	51
Figure 5.8: Health Records Screen	52
Figure 5.9: Diet Recommender Screen.....	53
Figure 5.10: Admin Dashboard Screen.....	54
Figure 5.11: Admin Reports Screen.....	55
Figure 5.12: Admin User Management Screen.....	56
Figure 5.13: Content Management Screen.....	57
Figure 5.14: Settings Screen	58

List of Tables

Table 3.1: Login/Signup	17
Table 3.2: Predict Diabetes Risk.....	18
Table 3.3: AI Chatbot for Diabetes Queries	18
Table 3.4: Retinopathy Detection	19
Table 3.5: Diabetic Foot Detection.....	20
Table 3.6: Health Record Management	20
Table 3.7: Test Result Analysis	21
Table 3.8: Emergency Support	21
Table 3.9: Personalized Nutrition Plans.....	22
Table 6.1: Manual Testing	63
Table 6.2: Functional Testing	66

Chapter 1

Introduction

1 Introduction

Diabetes mellitus is one of the most critical global health challenges of the twenty-first century. The International Diabetes Federation (IDF) estimates that the number of diabetic individuals will rise from 537 million in 2021 to over 700 million by 2045. This dramatic increase highlights the urgent need for innovative healthcare solutions, particularly those that leverage modern computing technologies and artificial intelligence (AI). Diabetes, being a chronic metabolic disorder, requires continuous monitoring, early diagnosis, timely medical intervention, lifestyle management, and regular screening for complications. However, traditional healthcare systems are not always equipped to deliver the level of continuous support that diabetic patients require, especially in developing regions.

In countries like Pakistan, diabetes is far more widespread than global averages. According to IDF reports, Pakistan ranks among the top nations with the fastest-growing diabetic population. Many patients lack access to structured medical care, specialized diagnostic facilities, and personalized consultation. Frequent hospital visits are costly and time-consuming, and patients often fail to maintain consistent health records or follow medical recommendations. Furthermore, diabetic complications such as neuropathy, nephropathy, retinopathy, and diabetic foot ulcers often go unnoticed in early stages due to lack of screening, eventually leading to irreversible damage.

Dr. Genie is developed to bridge these gaps through an AI-enabled, mobile-first healthcare platform. It integrates machine learning (ML) and deep learning (DL) models to predict diabetes risk, analyze retinal images for diabetic retinopathy, detect diabetic foot ulcers, and generate personalized dietary recommendations. The system supports users with a structured health record management module, emergency handling, a smart chatbot, and a user-friendly interface built using Flutter. With cloud-based synchronization through Firebase, the application ensures that patient data remains secure, accessible, and consistently updated.

1.1 Brief

Dr. Genie is a comprehensive AI-powered diabetes management application designed to

support early diagnosis, streamlined tracking, and informed decision-making for diabetic individuals. The application incorporates three primary diagnostic capabilities:

1. **Diabetes Prediction Model** – A machine learning model trained on clinical datasets to assess the likelihood of diabetes based on user-provided health parameters.
2. **Diabetic Retinopathy Detection** – A deep learning convolutional neural network (CNN)-based model that analyzes retinal fundus images to detect signs of retinopathy.
3. **Diabetic Foot Ulcer Detection (Neuropathy)** – A vision-based model using TensorFlow Lite to detect abnormalities in foot images relevant to neuropathy and ulceration.

In addition to diagnostic tools, Dr. Genie includes:

- Personalized diet and meal planning
- Blood glucose tracking with visual analysis
- Report and record management
- Appointment scheduling
- AI chatbot for guidance
- Emergency contact features

By unifying these modules, the application acts as a personalized digital healthcare assistant.

1.2 Relevance to Course Modules

The development of Dr. Genie synthesizes concepts from multiple computer science courses. It acts as a practical demonstration of how theoretical knowledge can be applied to create real-world software.

1.2.1 Web Technologies (CSC312)

The project extensively uses principles from Web Technologies:

- **RESTful API Integration:** Flask-based APIs serve ML models via HTTP requests.
- **JSON Serialization:** All ML inputs and outputs use JSON objects for communication.
- **Client-Server Architecture:** The mobile app communicates securely with backend ML services.
- **Firebase Web Services:** Authentication, Firestore, and Storage are cloud-hosted services accessed through web protocols.
- **CORS Policies:** Implemented in API endpoints to allow secure cross-origin

communication.

These topics allowed the development team to understand how distributed systems communicate and how cloud-based applications are deployed in a production-like environment.

1.2.2 Human-Computer Interaction (CSC356)

HCI concepts influenced the design of every screen:

- **User-Centered Design (UCD):** Interfaces were designed based on user needs and feedback.
- **Consistency & Standards:** The UI follows Material Design guidelines.
- **Error Prevention:** Forms include field validation and input constraints.
- **Accessibility:** High-contrast themes, large icons, and intuitive layouts help visually impaired users.
- **Feedback Mechanisms:** Loading indicators, animations, and success/error messages enhance user experience.

The application prioritizes usability and accessibility to ensure that users with varying technical backgrounds can comfortably use it.

1.2.3 Operating Systems (CSC322)

Several OS-level functionalities are integrated:

- **Camera and Storage Permissions:** Required for image capture and upload.
- **Thread Management:** ML model execution and HTTP requests run on separate threads to avoid freezing the UI.
- **Process Scheduling via `async/await`:** Ensures smooth user experience even during long-running model inference.
- **File I/O:** Temporary image caching for pre-processing.
- **Multitasking & Resource Management:** Optimizing memory when loading large ML models.

The project deeply aligns with OS principles regarding concurrency, device interaction, and memory management.

1.2.4 Database Systems I (CSC371)

Database concepts from this course form the foundation of storage design:

- **NoSQL Data Modeling:** Firestore organizes user data into structured collections and subcollections.
- **CRUD Operations:** Users can create, modify, read, and delete medical entries.

- **Document Relationships:** Health metrics, appointments, and prescriptions are linked using subcollections.
 - **Query Optimization:** Indexes improve performance for frequent queries.
 - **Security Rules:** Enforce user-based access control and authorization.
- The efficient data design contributes to performance, security, and scalability.

1.3 Project Background

In Diabetes management traditionally involves periodic clinical visits, manual tracking of glucose levels, and vague dietary recommendations. Patients often do not receive personalized guidance due to limited physician availability. Moreover, diabetic complications like retinopathy and neuropathy require specialized diagnostic equipment, placing them out of reach for many.

The emergence of mobile health (mHealth) apps has shown promise, but most available apps lack:

- AI-based prediction
- Image-based diagnostics
- Holistic integration of diet, records, and emergency features

Dr. Genie was conceptualized to fill these gaps by providing:

- A multi-model diagnostic system
- A real-time tracking and recommendation engine
- A secure and scalable cloud-based architecture
- An accessible interface suited for local populations

The project aims to transform how diabetic individuals interact with their health data, enabling early detection and informed decision-making.

1.4 Literature Review

In A detailed analysis of existing research highlights the effectiveness of AI in healthcare:

1.4.1 Machine Learning for Diabetes Prediction

- Logistic Regression, SVM, Random Forest, and Gradient Boosting can achieve **80–90% accuracy**.

- Feature importance studies show age, BMI, glucose levels, and family history strongly correlate with diabetes risk.

1.4.2 Deep Learning for Diabetic Retinopathy

- CNN models like MobileNetV2, ResNet50, and EfficientNet achieve:
 - **93–97% sensitivity**
 - **90–95% accuracy**
- Fundus image datasets such as APTOS, EyePACS, and Messidor are widely used.

1.4.3 Diabetic Foot Ulcer Detection

- CNN and MobileNet architectures can classify ulceration and infection.
- TensorFlow Lite models enable **on-device** inference.

1.4.4 Mobile Health App Effectiveness

- Studies show that digital interventions improve glycemic control.
- Personalized diet and educational content have long-term behavior impacts.

1.4.5 Analysis from Literature Review

From the reviewed studies:

- AI can significantly reduce undiagnosed diabetes cases.
- Early screening improves treatment outcomes.
- CNN-based image processing is reliable when datasets are diverse.
- Mobile apps improve patient compliance by **30–40%**.
- There is a clear research gap in **combined prediction + image analysis + diet planning** systems.

This reinforces the need for an integrated solution like Dr. Genie.

1.5 Methodology and Software Lifecycle for This Project

The development process followed an **Agile + Iterative Incremental** approach. Each module was developed in sprints, tested individually, and then integrated.

1.5.1 Key Phases

1. Requirement Analysis
2. UI/UX Prototyping
3. ML Model Training
4. API Development
5. Mobile Application Development
6. Integration Testing
7. Deployment
8. User Feedback Collection.

1.5.2 Rationale behind Selected Methodology

- ML models need iterative refinement
- UI requires user feedback loops
- Modular development supports parallel work
- Risks are minimized through continuous testing
- Feature expansion is easier

Chapter 2

Problem Definition

2 Problem Definition

Diabetes, being a chronic metabolic disorder, does not only influence blood glucose levels but affects nearly every organ system if not properly managed. One of the biggest challenges with diabetes is that it often progresses silently. Patients may remain unaware of early symptoms, leading to late detection of complications such as diabetic retinopathy, neuropathy, and nephropathy. Early diagnosis, regular monitoring, and personalized lifestyle management are essential to ensuring long-term health outcomes. However, the healthcare ecosystem—particularly in developing nations faces systemic challenges in providing holistic and affordable diabetic care.

This chapter outlines the central problem addressed by **Dr. Genie**, including the shortcomings of the current systems, project motivation, and the essential deliverables required to meet user needs. These problems form the foundation upon which the application's design, architecture, and implementation are constructed.

Context

Diabetes is no longer a condition restricted to age or lifestyle it has become a widespread, multifactorial condition. According to World Health Organization (WHO) statistics, a significant proportion of diabetic patients remain undiagnosed due to limited screening opportunities. Poor access to early diagnostic facilities, low health literacy, and lack of personalized care are major barriers.

Moreover, diabetic individuals are expected to maintain detailed records of:

- Blood glucose levels
- Diet patterns
- Exercise routines
- Medications
- Medical reports

- Doctor consultations

However, **manual record keeping** is error-prone, time-consuming, and rarely consistent. Patients often forget to record important parameters or misplace medical documents, leading to poor understanding of disease progression.

Adding to these challenges, diabetic patients must undergo regular screenings for complications. But specialized diagnostic procedures such as retinal scanning and foot ulcer assessment require:

- Expensive equipment
- Trained ophthalmologists or podiatrists
- Frequent clinical visits

For individuals in remote or underprivileged areas, such assessments become nearly impossible. Thus, there is a pressing need for a **digital, accessible, easy-to-use, AI-enhanced** solution to democratize diabetes screening and management.

2.1.1 Problem Statement

Access The core issues identified that motivated the development of **Dr. Genie** can be summarized as follows:

- **Lack of Accessible Early Screening Tools**

Many patients do not have access to early diabetes screening or diagnosis due to:

- Limited availability of medical facilities
- High consultation costs
- Lack of awareness

- **Fragmented Healthcare Applications**

Existing mobile apps focus on one or two aspects such as diet tracking or glucose logging; none provide a **consolidated platform** for:

- Diabetes prediction
- Retinopathy detection

- Foot ulcer detection
- Diet recommendations
- Record keeping
- Emergency assistance

- **Insufficient Monitoring for Diabetic Complications**

Diabetic retinopathy and neuropathy often go undetected until advanced stages.

This leads to irreversible conditions such as:

- Vision loss
- Chronic foot infections
- Limb amputation

- **Lack of Personalized and Data-Driven Diet Recommendations**

Diabetic diets vary based on:

- Age
- Weight
- BMI
- Physical activity
- Comorbidities

Generic diet plans fail to provide tailored advice.

- **Poor Health Record Management**

Manual records become:

- Disorganized
- Hard to track
- Easily misplaced

- **Limited Real-Time Support**

Patients often require quick information during hypoglycemic or hyperglycemic emergencies. Access to immediate guidance is rare.

- **Lack of Cloud-Based Synchronization**

Many apps store data locally, increasing the risk of:

- Data loss
- Missing medical history
- No cross-device continuity

- **Absence of ML/DL in Local Healthcare Apps**

Despite global advancements, Pakistan's medical apps rarely utilize:

- Machine learning predictions
- Image-based CNN diagnostics
- Automated trend analysis

These problems collectively justify the need for a comprehensive and intelligent digital companion for diabetic patients.

Context

Dr. Genie aims to deliver an all-in-one AI-powered diabetes management platform that integrates medical diagnostics, lifestyle recommendations, and real-time assistance.

The solution addresses the problems identified by providing:

2.1.2 Machine Learning–Based Diabetes Prediction

A predictive ML model analyzes user health parameters such as:

- Glucose levels
- BMI
- Age
- Blood pressure
- Family history

The model outputs:

- Percentage risk

- Risk level (low, medium, high)
- Guidance based on classification

This enables early detection without laboratory testing delays.

2.1.3 Deep Learning Image-Based Diagnosis

The system supports image-based screening for:

a. Diabetic Retinopathy

A CNN model evaluates retinal fundus images to detect signs such as:

- Microaneurysms
- Hemorrhages
- Exudates

b. Diabetic Foot Ulcers (Neuropathy)

A TensorFlow Lite model detects:

- Skin lesions
- Ulceration patterns
- Tissue discoloration

This reduces the dependency on specialists and equipment for first-stage screening.

2.1.4 Personalized Diet & Meal Recommendations

Diet planning considers:

- User profile
- BMI
- Activity level
- Preferences
- Glycemic load of food

A smart recommendation algorithm ensures that users receive meal options that

optimize glucose stability.

2.1.5 AI Chatbot for Guidance

An AI chatbot supports users with:

- Diabetes education
- Emergency steps
- Diet recommendations
- FAQs

This replaces the need for constant doctor consultation.

2.1.6 Blood Glucose Monitoring & Trends

Users can log glucose levels manually, enabling:

- Trend analysis
- Graph visualizations
- Automated alert triggers

2.1.7 Health Record Management

The app integrates:

- Upload & store reports
- Appointment details
- Prescriptions

All stored securely in Firebase.

2.1.8 Emergency Handling Module

A dedicated emergency system allows:

- Quick contact with caregivers or doctors
- Automated location sharing (optional)
- Display of emergency instructions

2.1.9 Cloud-Based Data Synchronization

Using Firebase ensures:

- Data backup
- Multi-device access
- Real-time synchronization

2.2 Deliverables and Development Requirements

Deliverables and development requirements.

2.2.1 Software Deliverables

- Fully functional Android mobile application (Flutter)
- Firebase-backed database and storage modules
- ML/DL models implemented via Flask APIs and TensorFlow Lite
- User-friendly dashboards and visual analytics
- Admin panel for management (optional)
- Automated diet and meal recommendation engine

2.2.2 Documentation Deliverables

- Final Year Project Report
- Requirement Specification Document
- Design Document (Diagrams + Architecture)
- Testing Documentation
- User Manual

2.2.3 Technical Requirements

- Flutter SDK
- Firebase Authentication
- Firestore Database
- Firebase Storage

- ML models (Python, TensorFlow, Keras)
- Flask REST API
- TFLite Interpreter
- Android Device with Camera Access

2.3 Current System

To better understand the need for Dr. Genie, we evaluate the shortcomings in existing solutions:

2.3.1 Existing System Weaknesses

- Apps like MySugr, Glucose Buddy focus only on tracking
- No image-based diagnosis
- No ML-based risk prediction
- Limited emergency handling
- No local adaptation for Pakistani users
- No integration of diet, ML, and medical records in one platform

2.3.2 Dr. Genie Improvements

- Unified ecosystem
- ML-powered predictions
- CNN-based medical image analysis
- Smart diet engine
- Localized UX/UI
- Firebase-backed secure storage

Chapter 3

Requirement Analysis

3 Requirement Analysis

Requirement analysis is a foundational phase of any software development lifecycle, serving as the backbone for system design, architecture, and implementation. For a sophisticated healthcare application like **Dr. Genie**, where accuracy, reliability, and safety are crucial, requirement analysis becomes even more essential. This chapter outlines all the functional and non-functional requirements, use cases, actors, and a detailed breakdown of system behavior. The purpose is to clearly define what the system must do (functional requirements) and how well it should perform (non-functional requirements), ensuring the application meets medical-grade reliability.

Use Cases Diagram(s)



Figure 3.1. Use Case Diagram

Detailed Use Case

Use case descriptions provide a detailed narrative of how the system interacts with various actors to accomplish specific tasks or goals. Each use case outlines the interactions between users, administrators, and Dr.Genie, capturing the functional requirements of the system. These descriptions serve as the foundation for understanding system behavior and ensuring all stakeholder needs are addressed during development. Use case descriptions bridge the gap between abstract requirements and tangible system designs. They provide clarity for developers, testers, and stakeholders.

3.1 Use Case Description

The table below indicates a comprehensive use case template filled in with an example drawn from the Online vehicle shipping management system. postconditions, main flow of events, alternate flows, and exception handling scenarios.

Table 3.1: Login/Signup

Use Case ID	01
Use Case Name	Login/Signup
Actors	User
Description	Allows an actor to log in or sign up to the system.
Pre-Condition	The admin must have a registered mail.
Post-Condition	The admin is logged into the system.
Normal Flow	1. The user accesses the login page. 2. The user enters their email and password. 3. The system validates the credentials. 4. The user is granted access to their account.
Alternative Flow	• A1 (Invalid credentials): Firebase returns error → Show error message + retry. • A2 (Weak password on signup): Client validation rejects → Prompt stronger password. • A3 (Email already used): Signup rejected → Offer password-recovery link.

	• A4 (Network unavailable): Offer offline cached-login if previously authenticated; otherwise show retry prompt. • A5 (Account locked / too many attempts): Temporarily block further attempts; show cooldown or contact support.
--	--

Table 3.2: Predict Diabetes Risk

Use Case ID	02
Use Case Name	Predict Diabetes Risk
Actors	User, Machine Learning Model
Description	User inputs symptoms/family history. The system predicts diabetes risk and provides results.
Pre-Condition	User must be logged in.
Post-Condition	System provides a risk assessment.
Normal Flow	1. User navigates to the Diabetes Prediction screen. 2. User enters required health details. 3. The system processes the data using ML. 4. The result is displayed to the user.
Alternative Flow	• A1 (Missing/invalid input): Client blocks submission and shows validation errors. • A2 (API timeout / server down): Show failure message; optionally queue request for retry; offer local fallback explanation. • A3 (Low-confidence result): API flags low confidence → Display cautionary note and recommend clinical follow-up. • A4 (Offline): Save input locally and allow user to submit when online (with local warning that prediction is pending).

Table 3.3: AI Chatbot for Diabetes Queries

Use Case ID	03
Use Case Name	AI Chatbot for Diabetes Queries
Actors	User, AI Chatbot

Description	User asks diabetes-related queries, and the chatbot provides answers.
Pre-Condition	User must have an internet connection.
Post-Condition	The chatbot provides an appropriate response.
Normal Flow	1. User opens the chatbot. 2. User asks a diabetes-related question. 3. The chatbot retrieves the answer and displays it.
Alternative Flow	• A1 (Unknown intent): Bot provides FAQ links, asks clarifying questions, or suggests contacting clinician. • A2 (Sensitive emergency detected): Bot triggers SOS/emergency flow (with user confirmation) if red-flag symptoms appear. • A3 (No network for advanced NLP): Fall back to rule-based responses stored locally.

Table 3.4: Retinopathy Detection

Use Case ID	04
Use Case Name	Retinopathy Detection
Actors	User, Machine Learning Model
Description	The system analyzes uploaded retinal images for diabetic retinopathy signs.
Pre-Condition	User must be logged in.
Post-Condition	System provides an analysis report.
Normal Flow	1. User uploads retinal image 2. System processes the image using deep learning. 3. The result is displayed to the user.
Alternative Flow	• A1 (Unacceptable image quality): Server or client rejects image → Prompt user to retake with guidelines (lighting, focus). • A2 (Large image / upload fail): Client compresses image automatically; if still fails, show error and allow retry. • A3 (Low confidence): Notify user that result is inconclusive and recommend clinic visit.

	• A4 (Privacy opt-out): User opts not to upload — allow local-only analysis if on-device model exists or decline operation.
--	--

Table 3.5: Diabetic Foot Detection

Use Case ID	05
Use Case Name	Diabetic Foot Detection
Actors	User, Machine Learning Model
Description	The system identifies potential foot complications through image analysis.
Pre-Condition	User must be logged in.
Post-Condition	System provides a detection report.
Normal Flow	1. User uploads foot image. 2. System processes the image using ML 3. The result is displayed to the user.
Alternative Flow	• A1 (Insufficient lighting / blur): Model returns low confidence → Prompt retake with instructions. • A2 (Device incapable / model load failure): Fall back to server-side analysis if user permits. • A3 (Offline but sync required): Save result locally and mark “unsynced” until connectivity restored. • A4 (False positive/negative concern): Provide “Second opinion” option to send image to clinician (with consent).

Table 3.6: Health Record Management

Use Case ID	06
Use Case Name	Health Record Management
Actors	User
Description	The system allows users to securely log and track vital health parameters.
Pre-Condition	User must be logged in.
Post-Condition	Health records are updated.

Normal Flow	1. User navigates to the Health Record Management screen. 2. User enters health parameters. 3. System logs and tracks the data.
Alternative Flow	• A1 (Invalid value / out-of-range): Client validation warns and asks for confirmation. • A2 (Offline entry): Save locally and sync when online; mark pending. • A3 (Duplicate entry): Detect possible duplicates and ask user to confirm overwrite or discard. • A4 (Privacy export request): User requests export → generate CSV/PDF.

Table 3.7: Test Result Analysis

Use Case ID	07
Use Case Name	Test Result Analysis
Actors	User
Description	The system provides insights and recommendations based on uploaded test results.
Pre-Condition	User must be logged in.
Post-Condition	System provides analysis and recommendations.
Normal Flow	1. User uploads test results. 2. System processes the data. 3. Insights and recommendations are displayed.
Alternative Flow	• A1 — Invalid File Format / Unsupported File Type • A2 — File Upload Failure / Network Interruption • A3 — Units / Reference Range Mismatch • A4 — Duplicate Test Detection

Table 3.8: Emergency Support

Use Case ID	08
Use Case Name	Emergency Support
Actors	User

Description	The system offers real-time guidance for managing diabetes-related emergencies.
Pre-Condition	User must be logged in.
Post-Condition	Emergency guidance is provided.
Normal Flow	1. User triggers the emergency support feature. 2. System provides real-time guidance. 3. System alerts contacts if required.
Alternative Flow	• A1 (No network): Use local cached emergency numbers and attempt call via cellular. • A2 (Call fails): Try backup contact numbers in priority order. • A3 (User cancels): Stop the flow and log cancellation. • A4 (Permissions denied): Show instructions to enable call/SMS/location permissions.

Table 3.9: Personalized Nutrition Plans

Use Case ID	09
Use Case Name	Personalized Nutrition Plans
Actors	User
Description	The system suggests meal plans and dietary recommendations tailored to individual preferences.
Pre-Condition	User must be logged in.
Post-Condition	Personalized nutrition plan is provided.
Normal Flow	1. User inputs preferences and health data. 2. System generates a personalized nutrition plan. 3. Plan is displayed to the user.
Alternative Flow	• A1 (Conflict with medical restrictions): Alert user and block unsuitable choices. • A2 (Sync conflict): If plan is edited on different device, show merge/conflict resolution options. • A3 (Notification disabled): Remind user to enable notifications for reminders.

3.2 Functional Requirements

Functional requirements define **what** the system must do.

Below is a detailed breakdown:

3.2.1 Authentication Module

- FR1: The system must allow users to register using email & password.
- FR2: The system must allow users to log in securely.
- FR3: The system must allow password recovery.
- FR4: The system must validate incorrect login attempts.

3.2.2 Diabetes Prediction Module

- FR5: The system must collect user inputs (age, BMI, glucose, BP, etc.).
- FR6: The system must send data to backend ML model.
- FR7: The system must display prediction probability.
- FR8: The system must show recommendations based on prediction outcome.

3.2.3 Retinopathy Detection Module

- FR9: The system must allow image upload.
- FR10: The system must pre-process images before sending to model.
- FR11: The system must process image via CNN model.
- FR12: The system must show severity level (Mild, Moderate, Severe).

3.2.4 Neuropathy/Foot Ulcer Detection Module

- FR13: App must accept foot images via camera/gallery.
- FR14: System must process image via TFLite model offline.
- FR15: System must show ulcer probability score.

3.2.5 Diet Recommendation Module

- FR16: The system must generate diet plans based on user profile.
- FR17: It must allow replacements/customization of meals.
- FR18: It must show nutritional values.

3.2.6 Chatbot Module

- FR19: The chatbot must answer basic diabetes-related queries.
- FR20: It must provide emergency instructions.
- FR21: Must support natural-language interaction.

3.2.7 Emergency Help

- FR22: System must store emergency contacts.

- FR23: App must trigger SMS/Call during emergency.

3.2.8 Health Record Management

- FR24: Users must upload medical reports/photos.
- FR25: System must store reports in Firebase Storage.
- FR26: Users must view previous medical records.

3.2.9 Glucose Monitoring

- FR27: User can enter blood glucose reading.
- FR28: System must generate graphs.
- FR29: System must detect abnormal patterns (optional).

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

- NFR1: Diabetes prediction response time < 2 seconds.
- NFR2: Retinopathy scan inference < 5 seconds (online).
- NFR3: Foot ulcer detection < 1 second (TFLite).
- NFR4: App should load dashboard in < 3 seconds.

3.3.2 Security Requirements

- NFR5: Firestore security rules must restrict user data.
- NFR6: Passwords must be encrypted via Firebase Auth.
- NFR7: Sensitive health data must use HTTPS/TLS.
- NFR8: Only authenticated users may access their records.

3.3.3 Usability Requirements

- NFR9: UI must follow Material Design.
- NFR10: System must be accessible for visually impaired users.
- NFR11: Buttons must have large tap areas.

3.3.4 Reliability Requirements

- NFR12: App uptime must be $\geq 99\%$.
- NFR13: Inference results must be consistent.
- NFR14: No data loss in cloud sync.

3.3.5 Maintainability Requirements

- NFR15: Code must follow MVC/MVVM structure.
- NFR16: ML models should be replaceable without UI changes.

3.3.6 Scalability Requirements

- NFR17: Firebase must handle increasing user traffic.
- NFR18: Backend API must support concurrent model requests.

3.3.7 Portability Requirements

- NFR19: App must run on Android 6.0 and above.
- NFR20: Future iOS compatibility must be possible.

Chapter 4

Design and Architecture

4 Design and Architecture

Dr. Genie is a mobile healthcare application designed to provide diabetic patients with a comprehensive and intelligent platform for managing their health. The system integrates multiple modules that function collaboratively to deliver predictive diagnostics, health monitoring, real-time support, and personalized care. Built on a client-server architecture, the application uses Flutter for cross-platform front-end development and Firebase with Node.js for back-end services, enabling secure data management and real-time synchronization. Machine learning models are embedded within the system to perform tasks such as diabetes prediction, retinopathy and foot ulcer detection, and automated test result interpretation. The AI-powered chatbot provides 24/7 assistance, ensuring users have access to medical guidance and emergency instructions at all times. With its modular and scalable design, Dr. Genie supports continuous feature expansion and cloud-based storage to ensure high performance, accessibility, and compliance with healthcare data regulations.

4.1 System Architecture

Dr. Genie follows a modular client-server architecture designed to support cross-platform mobile access, real-time data processing, and seamless integration of machine learning capabilities. The front end of the application is developed using Flutter, enabling a consistent user experience across both Android and iOS platforms. The backend utilizes Firebase for authentication, database management, and real-time synchronization, while Node.js handles API requests and business logic. Machine learning models developed in TensorFlow and PyTorch are either deployed through cloud-hosted APIs or integrated within the app for tasks such as image-based complication detection and risk prediction. All user data, including medical records and health inputs, is stored securely in a cloud-based database with encryption protocols to ensure privacy and compliance with HIPAA and GDPR regulations. The architecture is designed to allow independent development and testing of modules such as the Diabetes Predictor, Retinopathy Detector, and AI Chatbot, making it scalable, maintainable, and adaptable to future updates and medical advancements.

4.2 Data Representation [Diagram + Description]

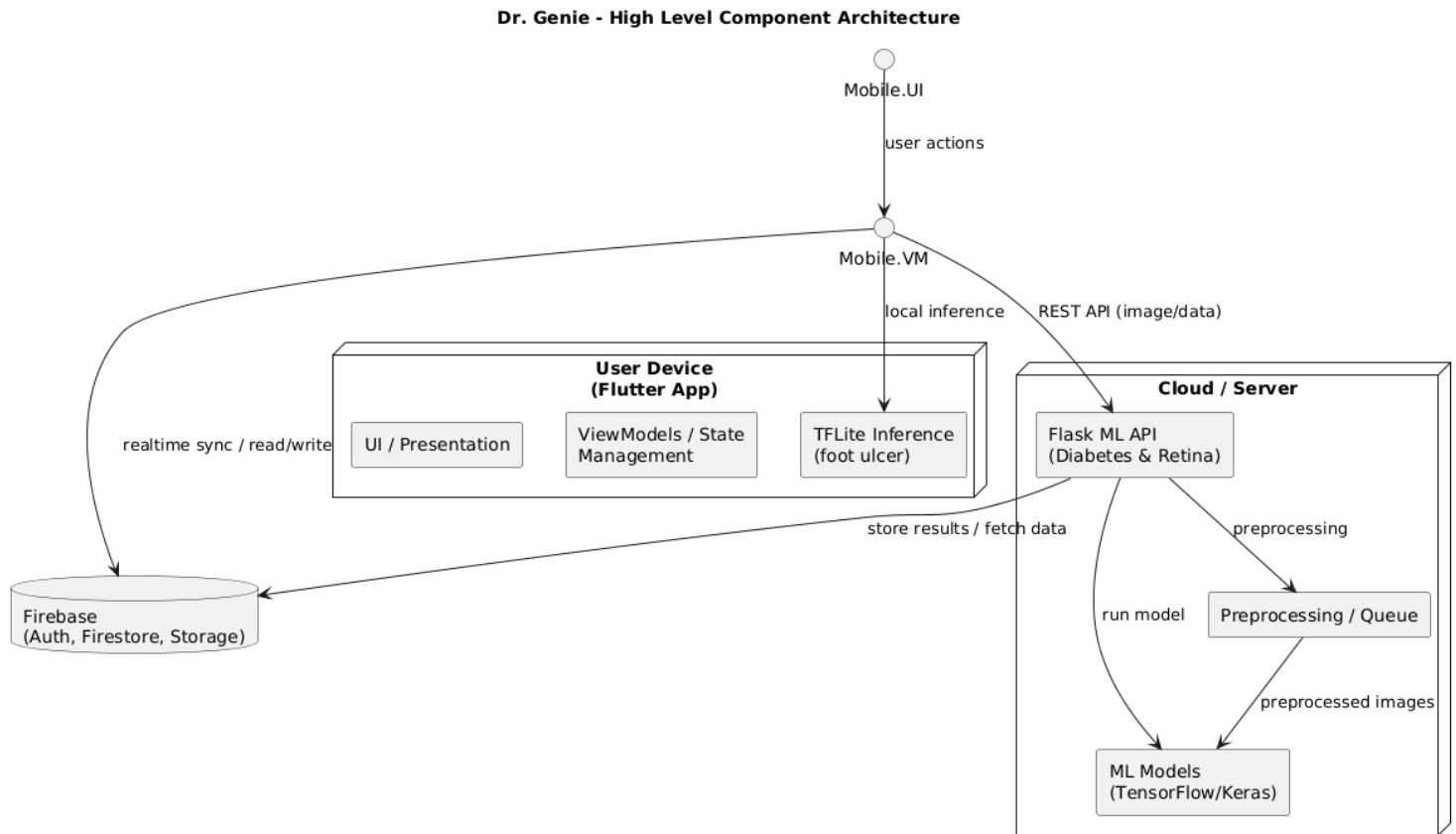


Figure 4.1: Architecture Diagram

4.3 Process Flow/Representation

The process flow of the **Dr. Genie** application outlines the comprehensive sequential interactions between the end-user and the system architecture. It demonstrates how distinct functional modules—ranging from user authentication to advanced machine learning inference—operate in coordination to deliver personalized, real-time diabetes management. The operational workflow can be categorized into the following critical stages.

1. Initialization and Secure Authentication

The workflow begins upon launching the application, where the system performs an initialization check. Users are prompted to verify their identity through **Firestore Authentication**, which handles secure registration and login procedures.

- **New Users:** Complete a profile setup requiring essential health demographics (age, weight, diabetes type) to calibrate the system for personalized results.

- **Returning Users:** Are granted immediate access via secure token validation, ensuring session continuity without repetitive logins.

2. Dashboard Navigation and Feature Selection

Post-authentication, the user is directed to the main **Dashboard**, which serves as the central navigational hub. From here, the user can select specific modules based on their immediate needs:

- **Diabetes Risk Prediction:** For assessing the likelihood of diabetes development or progression.
- **Image-Based Diagnostics:** Specifically for Diabetic Retinopathy and Foot Ulcer detection.
- **Lab Report Analysis:** For OCR-based or manual entry of blood test results.
- **Lifestyle Management:** Accessing the Nutrition Planner or Emergency Assistance.

3. Data Acquisition and Pre-processing

When a user interacts with a feature, the system initiates the data acquisition phase.

- **Numerical/Text Data:** Health metrics (e.g., glucose levels, BMI) entered for risk prediction are validated and normalized locally.
- **Image Data:** For retinopathy or foot ulcer detection, the user captures or uploads an image. The application performs image pre-processing (resizing, noise reduction, and formatting) to ensure the input is compatible with the underlying AI models.

4. Intelligent Processing (Inference Layer)

Once data is acquired, it is routed to the appropriate processing engine. This is a hybrid model involving:

- **Cloud-Based Inference:** Complex tasks, such as analyzing retinal scans via Convolutional Neural Networks (CNNs) or parsing complex natural language queries, are sent to the cloud server for heavy processing to ensure accuracy.
- **Local Processing:** Basic calculations and immediate data validation occur on the device to reduce latency. The system runs the input against trained Machine Learning models to generate diagnostic predictions, risk scores, or dietary recommendations.

5. Output Visualization and Feedback

The processed results are sent back to the User Interface (UI) for display. The system prioritizes clarity and actionable intelligence:

- **Visual Indicators:** Risk levels are displayed using color-coded gauges (e.g., Green for safe, Red for critical).
- **Detailed Reports:** Diagnostic modules provide probability percentages and suggested next steps (e.g., "Consult an ophthalmologist").
- **Dietary Plans:** The nutrition module generates structured meal plans based on the user's caloric requirements.

6. Interactive Support and Emergency Handling

Running parallel to these modules is the **AI Chatbot**. It remains accessible throughout the user journey to handle natural language queries, explain medical terminology, or guide the user through app features. In critical situations, the **Emergency Assistance** module can trigger alerts or provide immediate first-aid instructions based on the user's location and status.

7. Data Persistence and Synchronization

Finally, all interaction data, test results, and health logs are securely transmitted to the **Cloud Database (Firestore)**. This ensures that the user's medical history is preserved for future reference. The system synchronizes this data across sessions, allowing users to access their health analytics from multiple devices and ensuring a seamless, proactive care experience.

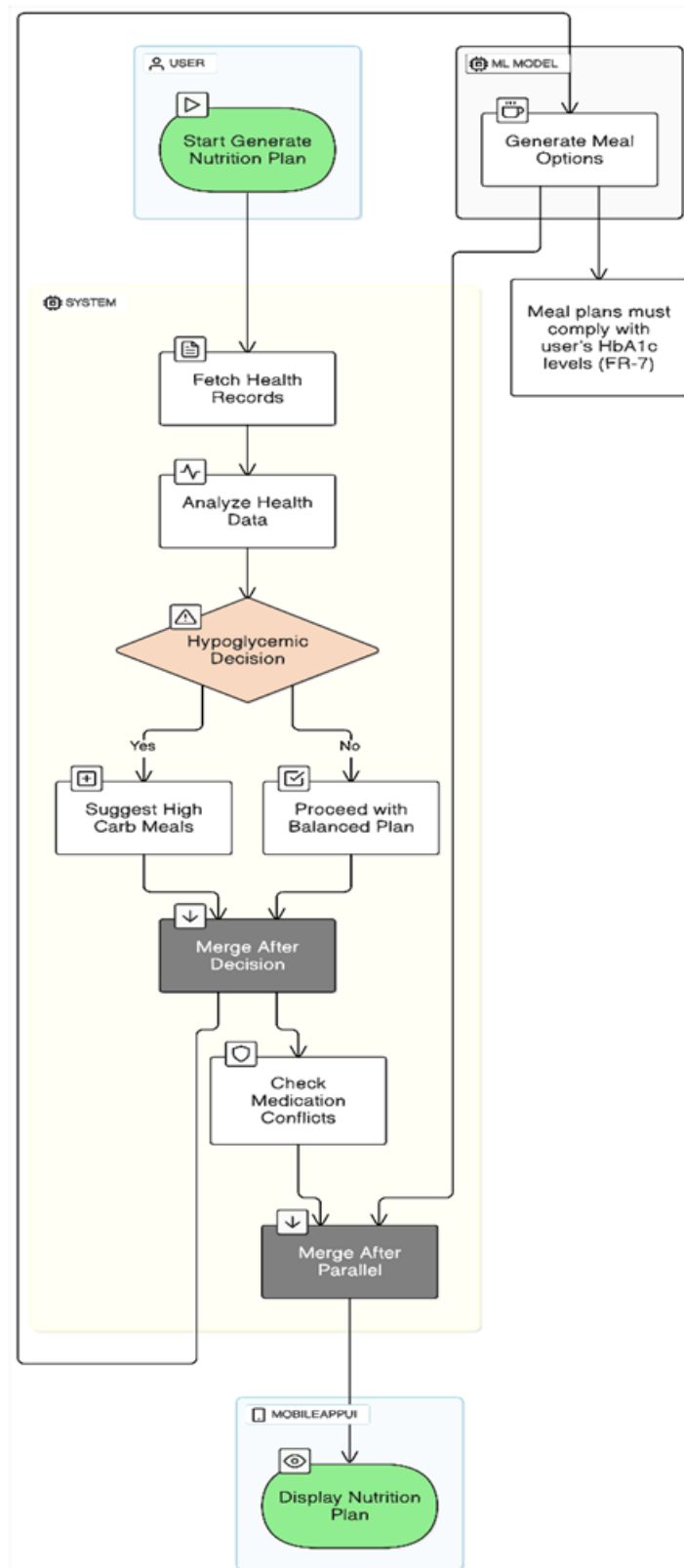


Figure 4.2: Flow Diagram

Design Models [along with descriptions]

The design models for Dr. Genie provide a visual and structural representation of how the system's components interact to achieve the intended functionalities. These models help in understanding the system's behavior, data flow, and internal structure, ensuring a reliable and maintainable architecture. The primary models include sequence diagrams and class diagrams for each core module, such as the Diabetes Predictor, Retinopathy Detector, Emergency Handler, and AI Chatbot. The sequence diagrams illustrate the step-by-step flow of interactions between users, the application interface, the backend logic, and integrated machine learning models, while the class diagrams define the structure of major components, their attributes, methods, and relationships. These design models are critical for guiding development, supporting modularity, and facilitating future enhancements or integration of new healthcare features.

4.4 Sequence Diagram

The following sequence diagram illustrates the interaction between the user and the Dr. Genie system when utilizing key functionalities like diabetes prediction, emergency assistance, and AI chatbot.

4.4.1 Sequence Diagram of Authentication

This sequence diagram provides a detailed explanation of the authentication workflow implemented in the Dr. Genie application. When a user enters their email and password on the login screen, the Flutter application sends the input credentials securely to Firebase Authentication for verification. Firebase Authentication, leveraging its robust cloud-based identity management system, then checks the provided credentials against its encrypted and secure user database. Upon verification, Firebase returns either a success response, indicating that the credentials are valid, or an error response if the credentials are incorrect or invalid. If the authentication is successful, the user is seamlessly redirected to the main dashboard of the application, granting them access to personalized features and data. However, if the authentication fails, an error message is displayed to the user, prompting them to re-enter their credentials. This entire flow is designed to ensure that the application offers secure, efficient, and reliable authentication, while keeping user data safe through the use of cloud-based identity management services provided by Firebase.

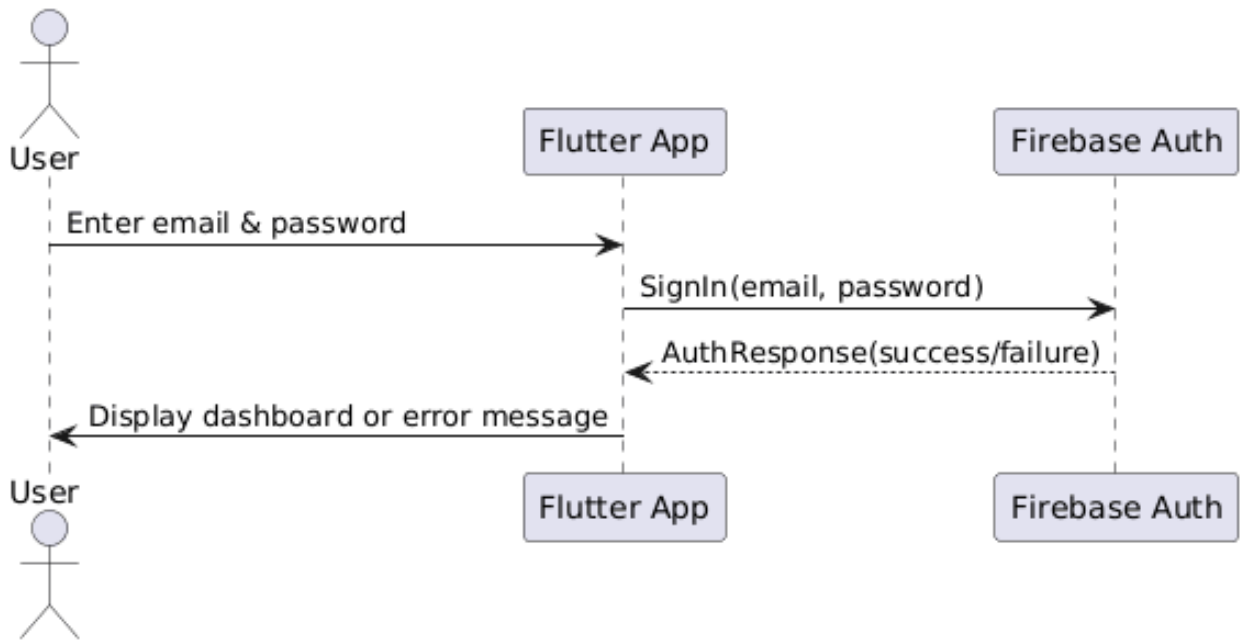


Figure 4.3: Sequence Diagram of Authentication

4.4.2 Sequence Diagram of Diabetes Prediction

This diagram illustrates the AI-driven diabetes risk prediction process. Users input their clinical values (BMI, glucose levels, blood pressure, etc.) into the app. The Flutter frontend sends this data to the Flask API, which processes the input and forwards it to the trained Random Forest model. The model calculates the probability of diabetes and returns a risk category (low, medium, or high). The Flask server stores the prediction in Firestore for record-keeping and sends the result back to the mobile app. The user then sees the prediction result along with tailored advice. This sequence demonstrates the communication between app, server, ML model, and database.

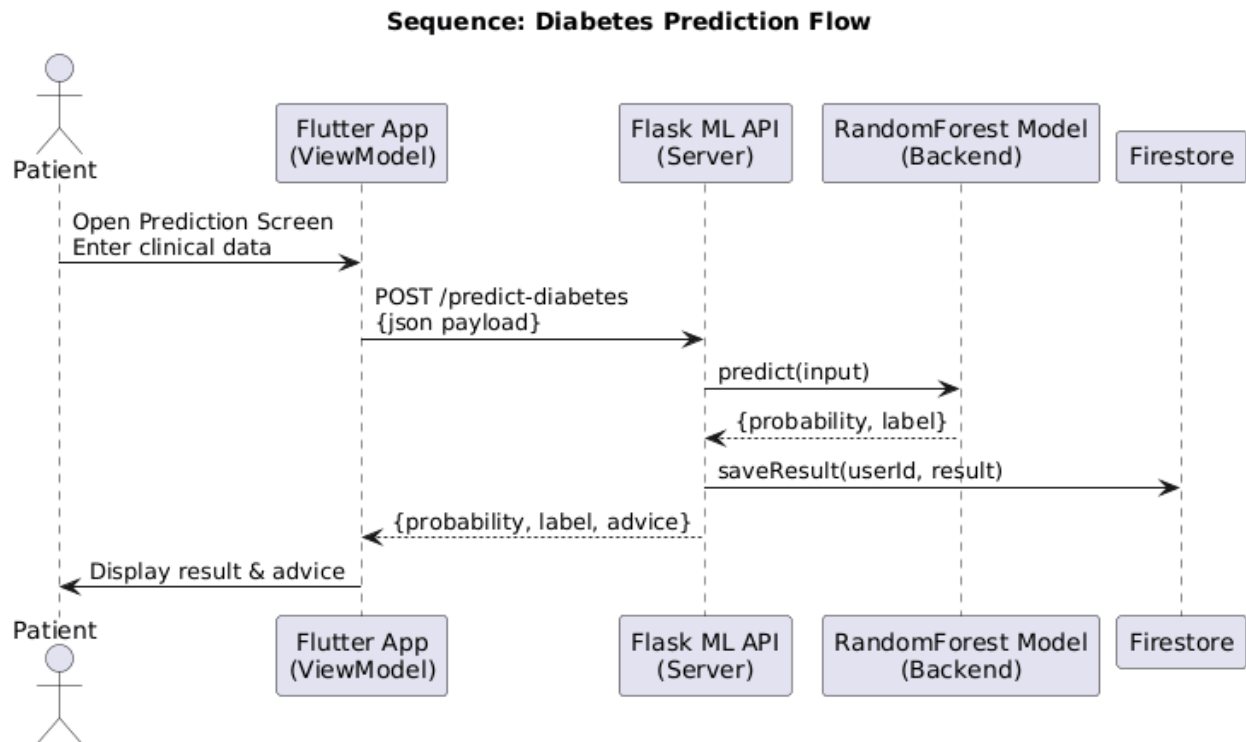


Figure 4.4: Sequence Diagram of Diabetes Prediction

4.4.3 Sequence Diagram of Retinopathy Detection

This diagram provides a comprehensive overview of the AI-driven diabetes risk prediction process within the application. Users are prompted to input their clinical values, such as BMI (Body Mass Index), glucose levels, blood pressure, and other relevant health metrics, directly into the app's interface. Once the user submits the data, the Flutter frontend securely transmits this information to the Flask API for further processing. The Flask API acts as an intermediary, receiving the clinical data and passing it along to a trained Random Forest machine learning model for analysis. The model, which has been trained on large datasets of medical information, calculates the likelihood or probability of the user developing diabetes based on their clinical inputs. After processing, the model returns a risk category (such as low, medium, or high) that reflects the user's potential risk of developing diabetes. To ensure proper record-keeping and future reference, the Flask server stores the prediction result, along with any relevant metadata, in Firestore, a cloud-based database. Once the result is securely saved, the Flask API sends the prediction outcome back to the mobile app, where the user is presented with their risk category and personalized advice tailored to their health data. This entire sequence highlights the seamless communication between the mobile app, the server-side Flask API, the machine learning model, and the cloud database, ensuring a smooth and efficient process from data input to result delivery.

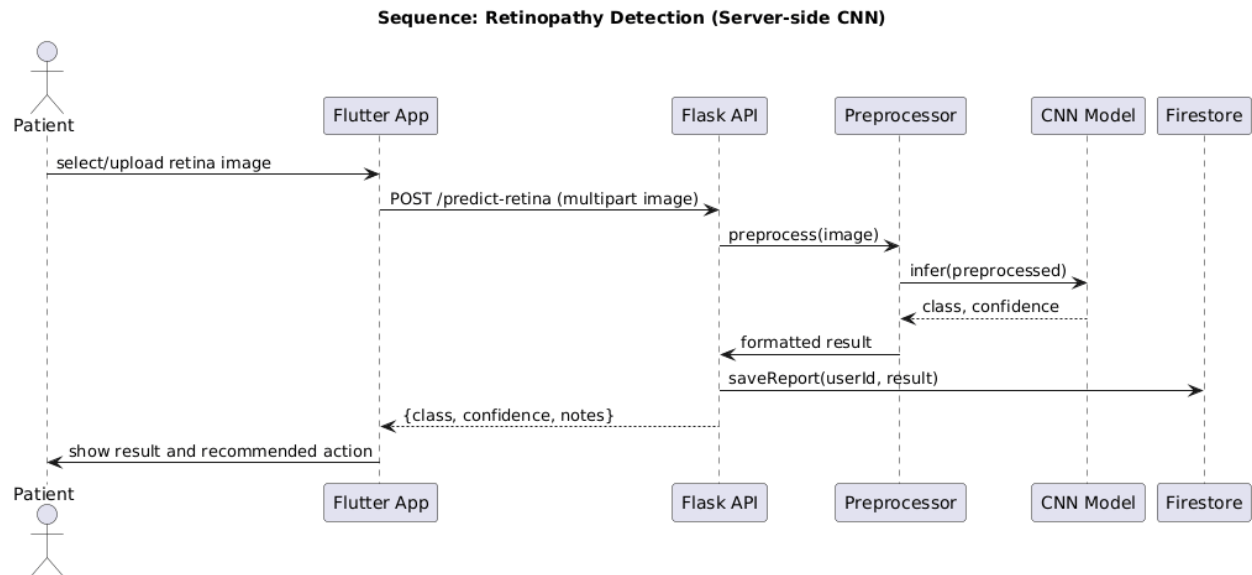


Figure 4.5: Sequence Diagram of Retinopathy Detection

4.4.4 Sequence Diagram of Neuropathy Detection

This diagram explains the foot-ulcer detection process, which runs entirely on the user's device for speed and offline capability. The user captures or selects an image of their foot. The Flutter application sends this image to the embedded TFLite interpreter. The MobileNetV2-based model performs real-time classification and returns the ulcer severity prediction. The app may store this result locally and later sync it to the cloud when internet is available. The prediction is instantly shown to the user, making the process fast and accessible even in low-connectivity areas.

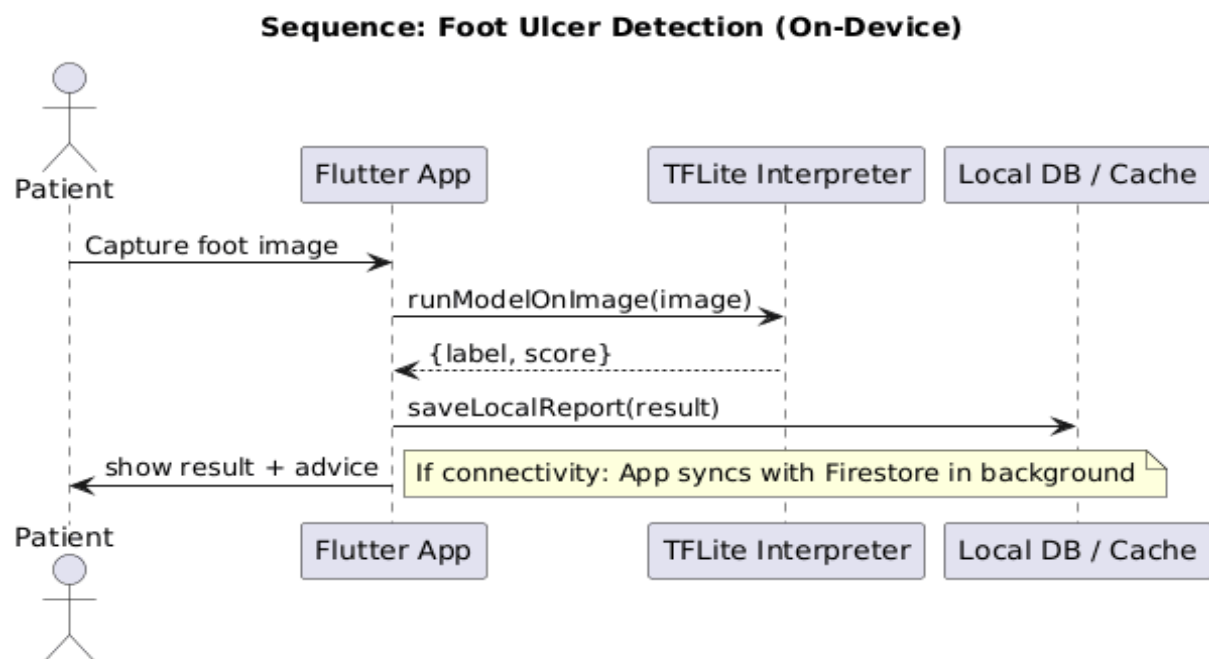


Figure 4.6: Sequence Diagram of Retinopathy Detection

4.4.5 Sequence Diagram of Meal Planner

This sequence describes how the AI-based diet recommender produces a personalized meal plan. When the user requests a new diet plan, the app retrieves their health profile from Firestore, including BMI, age, previous glucose readings, and dietary preferences. The data is sent to the Diet Engine, which uses a combination of rule-based filtering and ML-powered KNN matching to generate safe and diabetic-friendly meals. The engine returns a fully structured plan (breakfast, lunch, dinner) with calorie and nutrient detail. The app displays this plan in an organized layout. This process ensures personalized nutrition tailored to each user's health condition.

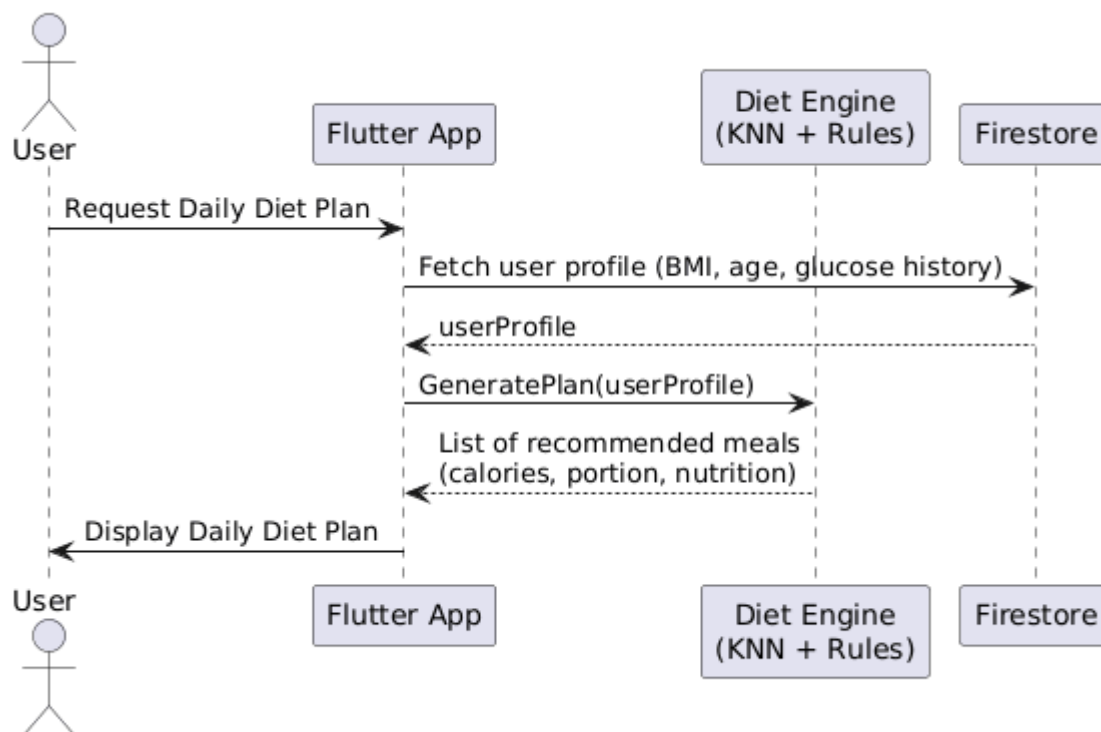


Figure 4.7: Sequence Diagram of Meal Planner

4.4.6 Sequence Diagram of AI Chatbot

This diagram outlines how the AI chatbot processes user queries. When the user types a message, the text is sent to the NLP engine inside the app. The engine tokenizes the user message, detects the intent using keyword patterns, and retrieves the appropriate response template from the response library. The formatted response is then returned to the chat interface and displayed to the user. This creates a conversational experience that provides health tips, symptom explanations, or general advice without needing internet-heavy LLM models. It helps users receive instant guidance in a familiar chat-style layout.

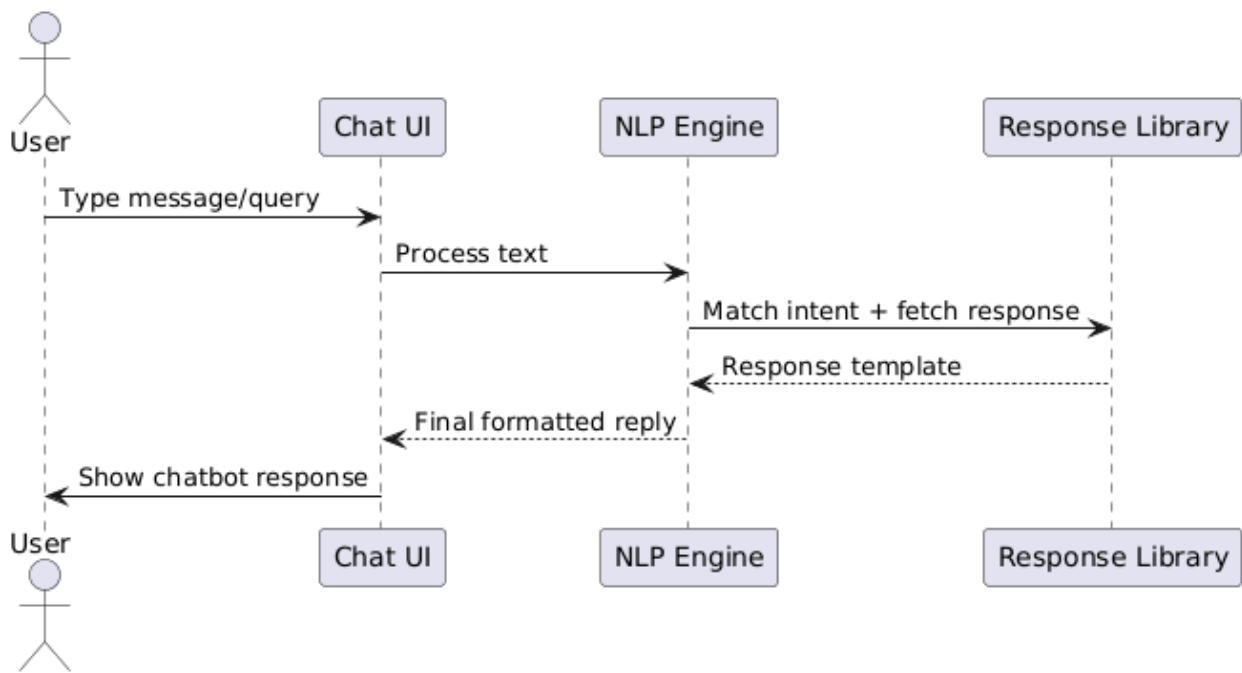


Figure 4.8: Sequence Diagram of AI Chatbot

4.5 Class Diagram

The attributes and methods of each class within the system can be further defined and customized based on your specific requirements, design goals, and the overall functionality of the application. These attributes represent the properties or characteristics of the class, while the methods define the actions or behaviors that the class can perform. Depending on the complexity of your implementation, additional details such as data types, access modifiers (public, private, etc.), and the relationships between classes (inheritance, composition, etc.) can be specified in the class diagram. By refining these elements, you can ensure that each class serves its intended purpose efficiently, aligns with the overall system architecture, and meets the unique needs of the project. This level of detail is crucial for establishing a clear and functional object-oriented design that supports both scalability and maintainability in the long term.

37

Chapter 5

Implementation

5 Implementation

Implementation is the core phase where the system design is transformed into a fully functional working product. This chapter covers the complete technical implementation of **Dr. Genie**, including the development environment, programming languages, frameworks, APIs, machine learning models, and the process of integrating different modules such as prediction engines, image analysis tools, data storage, UI components, and chatbot functionalities.

This chapter also provides pseudocode and algorithms for each major feature, demonstrating how the conceptual design was translated into executable code.

5.1 Development Environment & Tools

The implementation of Dr. Genie involved a diverse range of technologies and frameworks, each chosen for its performance, suitability, and compatibility with the overall system architecture.

5.1.1 Frontend Development (Flutter)

Flutter was chosen because of its robust UI capabilities and its suitability for cross-platform development.

Key Features Used

- StatelessWidget & StatefulWidget
- Provider & ChangeNotifier for state management
- Material Design components
- HTTP package for API communication
- TFLite plugin for on-device ML inference

Advantages

- Smooth animations
- High performance
- Fast development cycle with Hot Reload
- Universal UI consistency across devices

5.1.2 Backend Development (Flask API)

The Flask backend hosts and executes ML models, performing preprocessing and inference.

Why Flask?

- Lightweight
- Easy to integrate with ML models

- Supports JSON handling
- Can be deployed on cloud servers

5.1.3 Machine Learning & Deep Learning Models (Python)

Python served as the core language for building ML models due to its extensive ecosystem of ML libraries.

Libraries Used

- TensorFlow
- Keras
- Scikit-learn
- OpenCV
- Pandas
- NumPy

Models were trained separately and then exported for:

- REST API inference (diabetes prediction)
- TFLite inference (neuropathy detection)

5.1.4 Firebase Integration

Firebase provided cloud services for data storage, authentication, and real-time updates.

Firestore Components Used

- Firebase Authentication
- Firestore NoSQL Database
- Firebase Storage
- Firebase Cloud Messaging (optional)

5.1.5 On-Device Inference (TensorFlow Lite)

The TFLite model allowed real-time image classification with minimal resource consumption.

5.2 Core Algorithms

This section explains in detail the core algorithms powering the major components of **Dr. Genie**, including diabetes prediction, retinal disease detection, neuropathy detection, diet recommendation, and the personalized health insight engine. These algorithms form the computational backbone of the system and significantly contribute to the accuracy, intelligence, and reliability of the final application.

5.2.1 Diabetes Prediction Algorithm

The diabetes prediction module uses the **Random Forest Algorithm**, a widely used ensemble

learning technique for classification tasks. This model is trained on the **PIMA Indians Diabetes Dataset**, which contains eight physiological and biochemical attributes related to diabetes risk factors.

Why Random Forest?

- Handles nonlinear relationships
- Robust to noise and outliers
- Provides high accuracy with low overfitting
- Works well with small-to-medium datasets

Input Features

- Glucose Level
- BMI
- Age
- Blood Pressure
- Insulin Level
- Skin Thickness
- Pregnancies
- Diabetes Pedigree Function

Working Mechanism

1. Random subsets of the dataset are selected.
2. Multiple Decision Trees are trained in parallel.
3. Each decision tree votes for a class (diabetic or not).
4. The majority vote decides the final outcome.
5. Feature importance scores help interpret model reasoning.

Mathematical Overview

Random Forest uses the **Gini Impurity** measure:

$$G = 1 - \sum_{i=1}^c p_i^2$$

Where:

- p_i is the probability of class i
- C = number of classes

The final prediction is computed as:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \dots, T_n(x))$$

Where $T_j(x)$ is the j -th decision tree's prediction.

Advantages

- Very stable predictions
- Easily interpretable feature rankings
- Can handle missing/irregular medical data

Pseudo Code

Input: BMI, Glucose, Age, BloodPressure, Insulin, SkinThickness

Output: DiabetesRiskScore, RiskCategory

Begin

Collect user inputs

Normalize input values

Load trained RandomForestModel

prediction = RandomForestModel.predict(input)

probability = RandomForestModel.predict_proba(input)

If probability < 0.3:

RiskCategory = "Low"

Else if probability < 0.7:

RiskCategory = "Medium"

Else:

RiskCategory = "High"

Return probability, RiskCategory

End

Workflow of Model

1. User enters health parameters.
2. The data is validated.
3. Values are normalized.
4. Flask API receives inputs and runs inference.
5. The prediction result is returned as JSON.

5.2.2 Retinopathy Detection Algorithm (Convolutional Neural Network)

Diabetic Retinopathy (DR) detection relies on a **Convolutional Neural Network (CNN)** trained on APTOS/EyePACS datasets. Retinal images are classified into 5 stages:

- No DR
- Mild
- Moderate
- Severe
- Proliferative DR

Why CNN?

CNNs excel at:

- Image classification
- Learning spatial features
- Detecting subtle medical patterns invisible to the naked eye

Model Pipeline

1. **Image Preprocessing**
 - Resizing to 224×224
 - Normalization
 - CLAHE enhancement
2. **CNN Layers**
 - Convolution (feature extraction)
 - ReLU activation
 - MaxPooling
 - Dropout (for regularization)

3. Dense Layers

- Combine extracted features

4. Softmax Output

- Probability distribution over classes

Key Mathematical Operation

Convolution:

$$(feature\ map)(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot K(m, n)$$

where:

- X = image
- K = kernel/filter

Advantages

- High sensitivity and specificity
- Scales well to large datasets
- Strong generalization in medical imaging

Pseudocode

Input: FundusImage

Output: DR_SeverityLevel

Begin

Read image file

Resize to 224x224

Normalize pixel values

Expand dimensions (batch size)

Load CNN model

prediction = model.predict(image)

class = argmax(prediction)

Map class to severity:

0 -> No DR

1 -> Mild

2 -> Moderate

3 -> Severe

4 -> Proliferative DR

Return severity level

End

5.2.3 Neuropathy (Diabetic Foot Ulcer) Detection Algorithm (MobileNetV2 + TFLite)

For neuropathy detection, a **MobileNetV2** model is used. The model is trained on DFU datasets and converted to **TensorFlow Lite** for on-device inference.

Why MobileNetV2?

- Lightweight
- Fast inference on smartphones
- High accuracy with low computational cost

Key Concepts

- Depthwise-Separable Convolutions
- Inverted Residuals
- Linear Bottlenecks

MobileNetV2 Block Operation

$$y = x + F(x)$$

where $F(x)$ is the non-linear transformation block (inverted residual).

Output

Classifies images into:

- No ulcer
- Early-stage ulcer
- Severe ulcer

This helps users identify foot complications early.

Pseudocode

Input: FootImage

Output: UlcerProbability, Label

Begin

Load TFLite interpreter

Resize image to model input shape

Normalize pixels

Run inference using interpreter

Get output probability

If probability > 0.5:

Label = "Ulcer Detected"

Else:

Label = "Normal"

Return probability, Label

End

5.2.4 Diet Recommendation Algorithm (Hybrid ML + Rule-Based Engine)

The diet module uses a **Hybrid Algorithm**:

- 1. Rule-Based Filtering**

- Based on diabetes guidelines (e.g., AHA, ADA)
- Removes foods with high glycemic index

- 2. Machine Learning (K-Nearest Neighbors)**

- Matches meals with similar nutritional profiles

- 3. Calorie Optimization (Harris-Benedict Equation)**

- Calculates user's BMR and daily caloric need

- **Harris-Benedict Equation**

For males:

$$BMR = 88.36 + (13.4 \times W) + (4.8 \times H) - (5.7 \times A)$$

For females:

$$BMR = 447.6 + (9.2 \times W) + (3.1 \times H) - (4.3 \times A)$$

Meal Matching Using KNN

Meals with similar

- Calories

- Carbs
- Fiber
- Glycemic index

are grouped and recommended.

Advantages

- Personalized diet
- Adaptive to user conditions
- Nutritionally optimized

Pseudocode

Input: userProfile

Output: DailyMealPlan

Begin

Calculate basal metabolic rate (BMR)

Determine total daily calorie need

Fetch food items from database

Filter meals based on user's restrictions

Generate balanced meals:

- 30% breakfast

- 40% lunch

- 30% dinner

Ensure glycemic load < threshold

Return meal plan

End

5.3 External APIs

Several APIs were integrated:

5.3.1 Prediction API (Flask)

Endpoint: /predict-diabetes

Method: POST

Input: JSON

Output: risk_percentage, risk_category

5.3.2 Retinopathy Detection API

Endpoint: /predict-retina

Method: POST (Image upload)

5.3.3 Chatbot API

Uses simple NLP text classification for guidance.

5.4 User Interface

The user interface of the Dr.Genie mobile application is designed to be clean, minimal, and user-friendly, keeping accessibility and simplicity in mind. The app provides users with intuitive navigation and clearly labeled options for interacting with the core features, including report-based health suggestions and an AI-powered chatbot.

The main screens and their descriptions are as follows

Dashboard Screen

The dashboard serves as the central navigation hub.
Modules:

- Prediction
- Detection
- Diet
- Chatbot
- Glucose logging
- Emergency

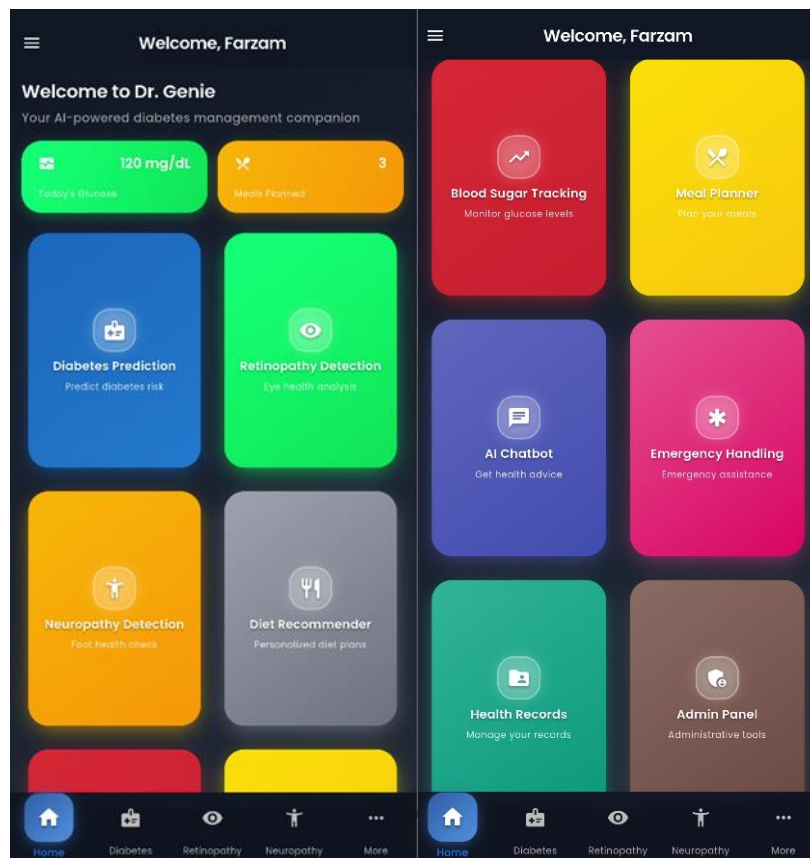


Figure 5.1: DashBoard Screen

Authentications Screens

These screens handle user registration and secure access to the application. Using Firebase Authentication, the system allows users to sign up with their email, log in securely, and reset their password when needed. The UI follows a clean and intuitive layout with input fields, validation messages, and actionable buttons. These screens ensure that only authorized individuals access personal health data, maintaining privacy and system security.

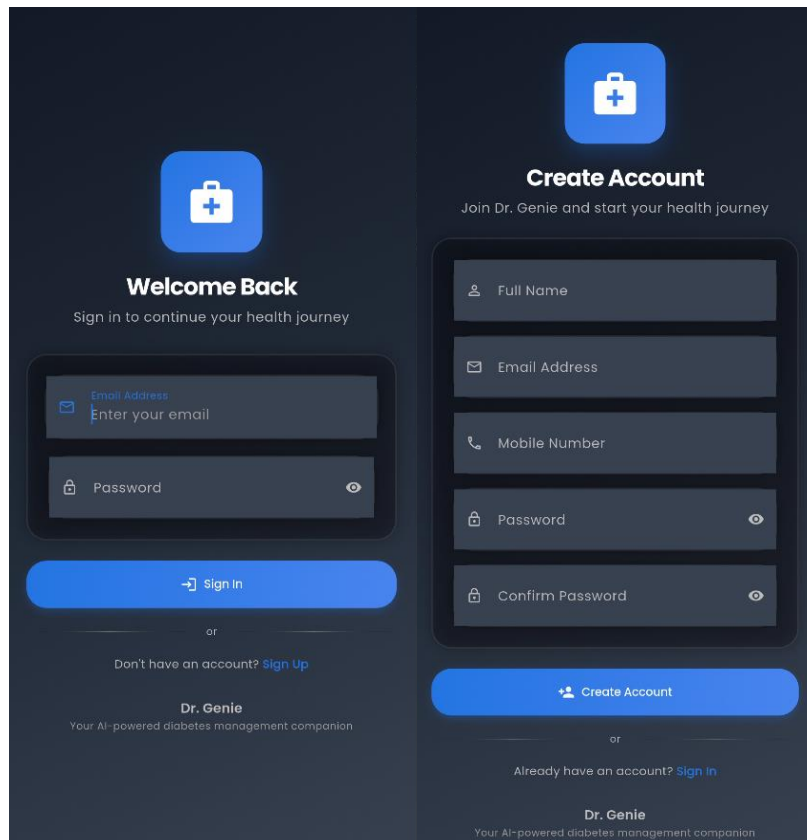


Figure 5.2: Authentication Screens

Diabetes Prediction (Random Forest)

This screen collects user health information such as glucose levels, BMI, age, blood pressure, and other key clinical attributes. Once submitted, the data is sent to the backend Random Forest classifier, which predicts the user's diabetes risk and provides a percentage-based likelihood. The result is displayed clearly with risk category labels and personalized recommendations. The interface is minimal and form-based, ensuring ease of use even for patients unfamiliar with technology. This feature offers a quick, AI-driven assessment tool for identifying potential diabetic conditions.

← Diabetes Prediction ↻

This tool predicts the likelihood of diabetes based on symptoms and demographic data. Please fill in the form below accurately for the best results.

Patient Name

Age

Gender ▾

Polyuria ▾

Polydipsia ▾

Sudden Weight Loss ▾

Weakness ▾

Polyphagia ▾

Genital Thrush ▾

Visual Blurring ▾

Home Diabetes Retinopathy Neuropathy More

Figure 5.3: Diabetes Prediction Screen

Retinopathy Detection Screen

This screen is designed for uploading or capturing retinal images to assess the presence of diabetic retinopathy. Using a trained CNN model deployed on the backend server, the application processes the image and categorizes the retinopathy stage. The interface includes an image preview, upload buttons, and a results section showing the predicted severity and confidence score. This feature enables early screening for a vision-threatening condition that often remains unnoticed until advanced stages, thus supporting preventive diabetic care.

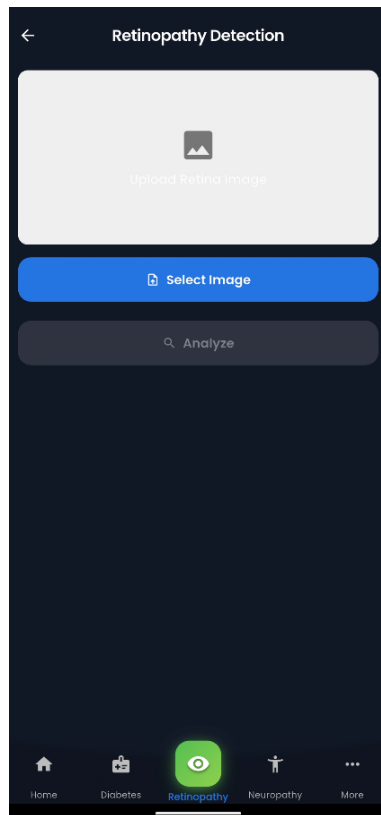


Figure 5.4: Retinopathy Detection Screen

Neuropathy Screen

This screen enables users to either upload existing images or capture new photos of their feet using the device camera for the early detection of diabetic neuropathy or potential foot ulcers. By leveraging an on-device TensorFlow Lite (TFLite) deep learning model, the system analyzes the captured image directly on the user's device, ensuring fast performance and enhanced data privacy. The model examines visual patterns such as skin discoloration, wounds, or tissue abnormalities to identify the possible presence of ulcers or tissue damage. Once the user selects or captures an image, the application performs real-time analysis and processes the image to determine the predicted severity level, which is then clearly displayed on the screen. The user interface is designed to be intuitive and user-friendly, featuring clear instructions, visual indicators, and guided steps to help users navigate the detection process with ease, even for non-technical users. Overall, this feature empowers diabetic patients to proactively monitor their foot health, detect complications at an early stage, and seek timely medical intervention, thereby reducing the risk of severe outcomes such as infections or amputations.

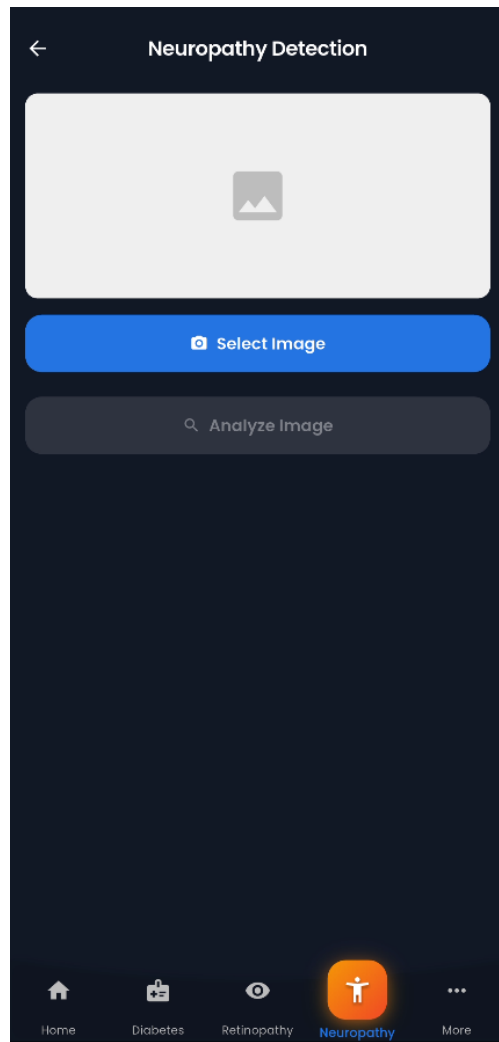


Figure 5.5: Neuropathy Detection Screen

AI Chatbot Screen

This screen enables users to communicate directly with the AI-powered health assistant that is seamlessly integrated into the application. Through this interface, users can type and submit questions related to a wide range of health topics, including symptoms they may be experiencing, prescribed medications, dietary and nutrition choices, daily routines, lifestyle habits, or general health and wellness concerns. The chatbot processes these inputs and responds intelligently and almost instantly, providing relevant, context-aware information and suggestions that create a smooth and engaging conversational experience. The interface is designed to resemble a familiar messaging application, with clearly separated and visually distinct user and bot messages to enhance readability and

ease of interaction. By offering quick, accessible, and informative responses, this feature allows users to receive basic medical guidance and health-related insights without the immediate need for in-person consultations. As a result, it improves overall user engagement, promotes health awareness, and supports users in making more informed decisions about their well-being.

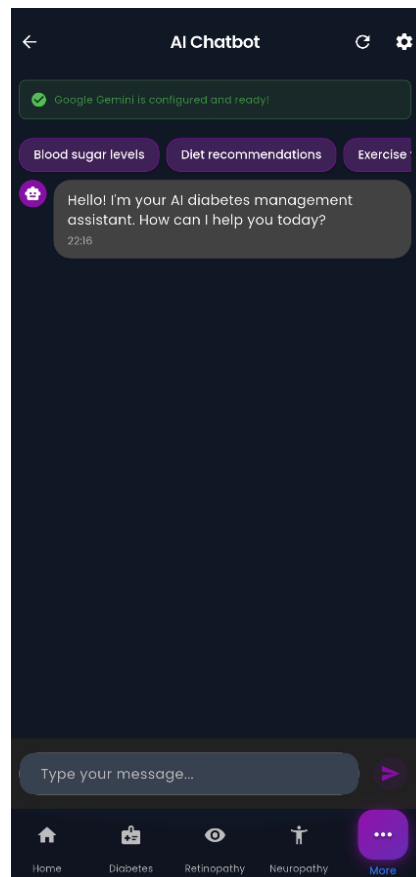


Figure 5.6: Ai ChatBot

Meal Planner Screen

The Meal Planner screen enables users to efficiently organize and plan their daily meals according to their individual dietary needs and health goals. It offers clearly structured meal categories, including breakfast, lunch, dinner, and snacks, each populated with recommended food items specifically tailored for individuals managing diabetes. For every suggested meal or food item, users can access detailed nutritional information such as calorie counts, carbohydrate content, and suggested serving sizes to support informed decision-making. The screen features a clean, intuitive, and visually organized layout that makes it easy for users to browse options, customize their meal plans, and maintain a healthy and balanced lifestyle. By providing consistent guidance and practical meal suggestions, this feature helps users adhere to recommended nutrition guidelines, regulate blood sugar levels, and manage their condition more

effectively over time.

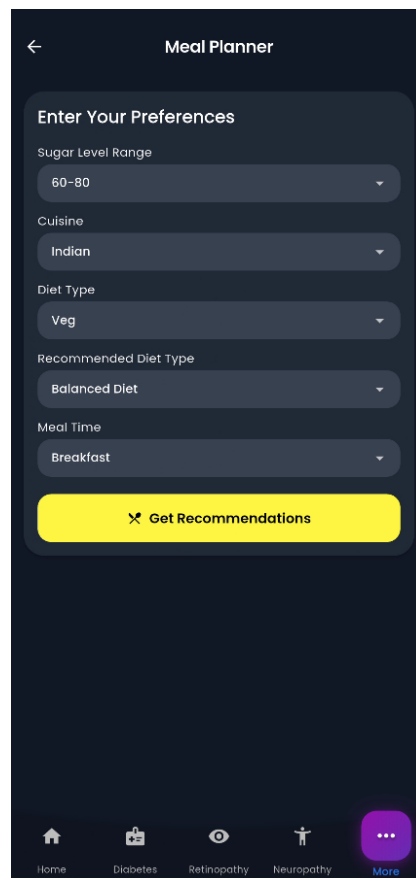
The image shows a mobile application screen titled "Meal Planner". At the top left is a back arrow icon. Below the title is a section "Enter Your Preferences" containing five dropdown menus: "Sugar Level Range" (set to 60-80), "Cuisine" (set to Indian), "Diet Type" (set to Veg), "Recommended Diet Type" (set to Balanced Diet), and "Meal Time" (set to Breakfast). A yellow button with a fork and knife icon and the text "Get Recommendations" is positioned below these menus. At the bottom of the screen is a navigation bar with five icons: a house for "Home", a briefcase for "Diabetes", an eye for "Retinopathy", a person for "Neuropathy", and a purple circle with three dots for "More".

Figure 5.7: Meal Planner Screen

Health Records Screen

This screen provides a centralized and secure location for storing, managing, and viewing the user's medical records within the application. Users can upload a variety of medical documents, including reports, prescriptions, laboratory test results, and relevant medical images, either directly from their device storage or by capturing new images using the camera. Each uploaded file is securely stored in Firebase Storage and systematically linked to the user's profile in Firestore, ensuring both data safety and easy retrieval. The interface presents the files in a well-organized list or grid format, complete with timestamps, file names, and categorized labels to help users quickly locate specific records. By maintaining a comprehensive digital medical history, this feature enables users to conveniently track their health progress over time and easily share accurate and up-to-date

medical information with healthcare providers when needed.

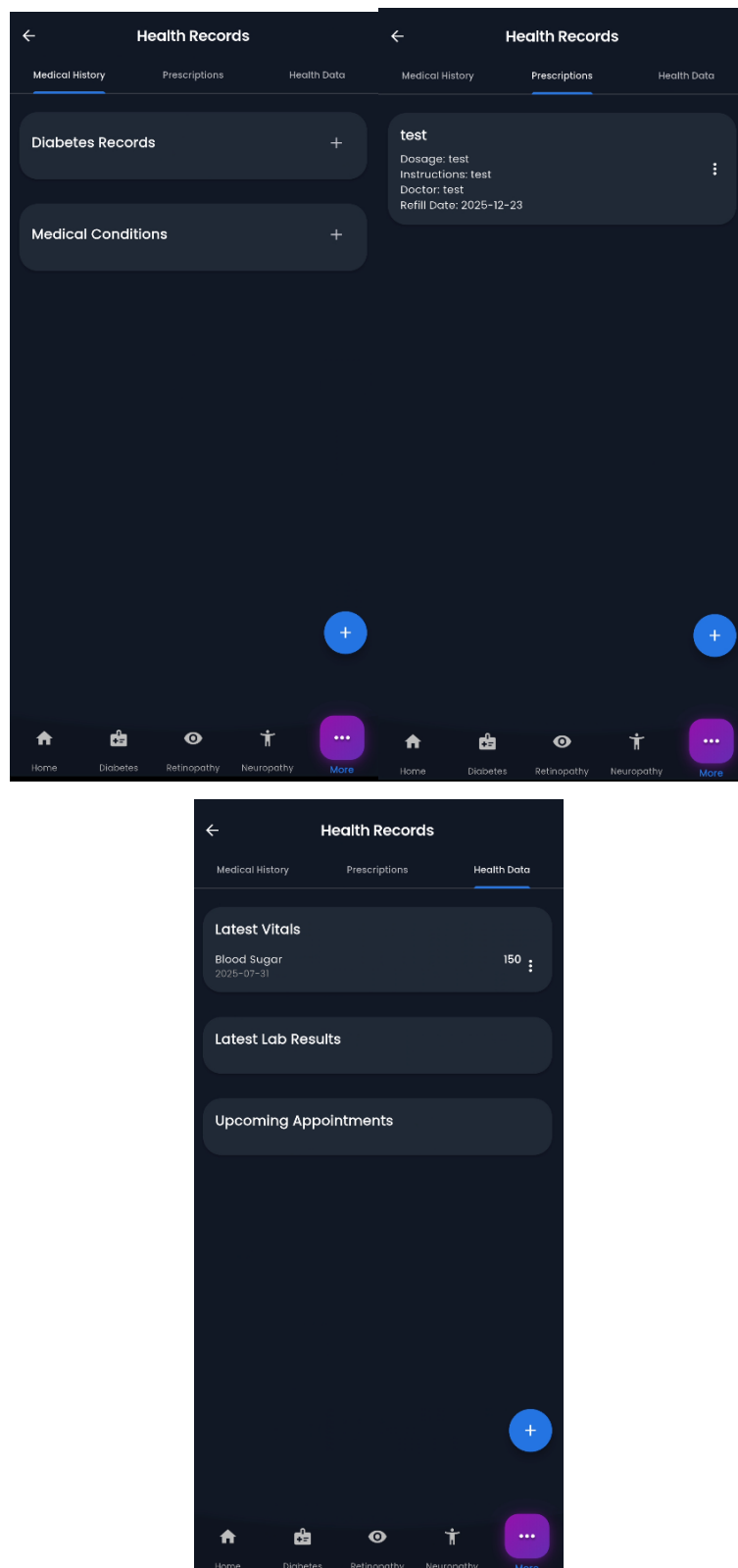


Figure 5.8 Health Records Screen

Diet Recommender Screen

This screen presents AI-generated diet recommendations based on the user's personal health profile, BMI, caloric needs, and glycemic targets. Using a hybrid rule-based and ML-assisted engine, the system suggests meals that are safe, nutritious, and diabetic-friendly. Recommendations include portion sizes, calorie counts, and nutritional breakdowns. The UI is clean and engaging, allowing users to browse suggestions easily. This feature supports healthier eating habits and long-term lifestyle improvement.

The screenshot displays the 'Diet Recommender' screen with a dark theme. It features three main sections: Personal Information, Health Information, and Lifestyle Information. The bottom navigation bar includes icons for Home, Diabetes, Retinopathy, Neuropathy, and a 'More' button.

Personal Information	
Age	Gender
30	Male
Height (cm)	Weight (kg)
170.0	70.0

Health Information	
Chronic Disease	
None	
Systolic BP	Diastolic BP
120	80
Cholesterol	Blood Sugar
200	100

Lifestyle Information	
Genetic Risk Factor	☐
Exercise Frequency (days/week)	
<input type="range"/>	
Sleep Hours	
<input type="range"/>	

Figure 5.9: Diet Recommender Screen

Admin Dashboard Screen

The Admin Dashboard screen provides a high-level overview for administrators to monitor system activity and manage platform data. It displays key metrics such as number of users, uploaded reports, model predictions, and platform interactions. The dashboard is visually structured with cards, charts, and quick navigation tabs. This feature allows admins to efficiently oversee system performance and user engagement.

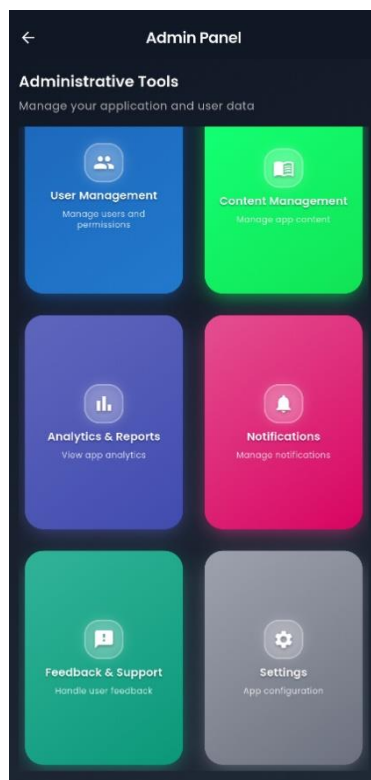


Figure 5.10: Admin Dashboard

Admin Reports Screen

This screen allows administrators to efficiently view, verify, and manage all medical reports uploaded by users within the system. Administrators can review a wide range of medical data, including retina scans, foot images, and automatically generated predictions produced by the AI models, in order to ensure their quality, accuracy, and consistency. The interface is equipped with advanced search filters, sorting options, and detailed report preview features that enable

admins to quickly locate specific records and examine them in greater detail. By providing comprehensive oversight and validation tools, this feature helps maintain transparency, correctness, and accountability in medical data handling, while ensuring that the information stored and presented within the application meets required standards.

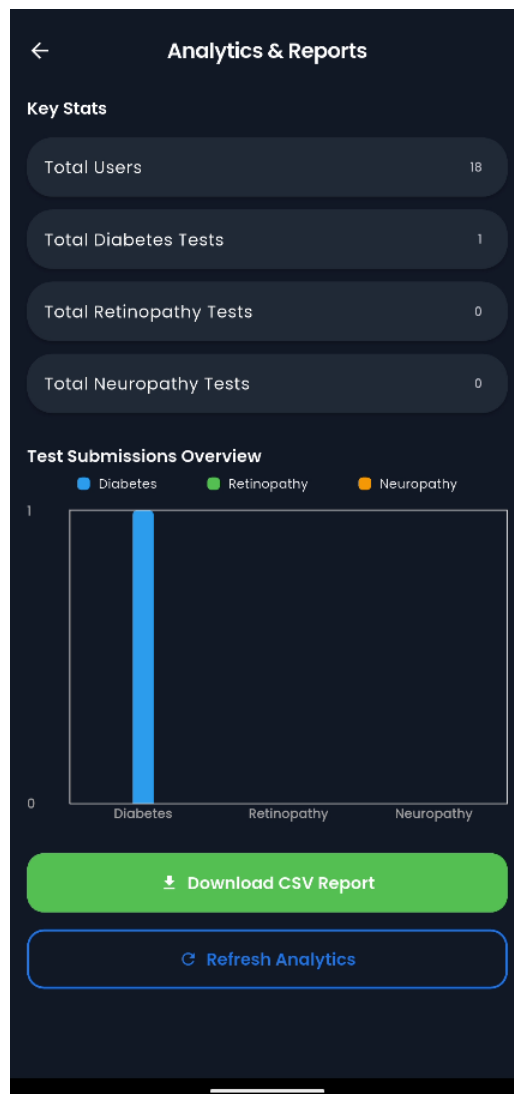


Figure 5.11: Admin Reports Screen

Admin User Management Screen

This screen provides tools for administrators to manage user accounts, including viewing user profiles, disabling accounts, or resetting access when required. Admins can search users by ID, email, or name, and view associated activity summaries. The design emphasizes clarity and administrative control, supporting system security and platform governance.

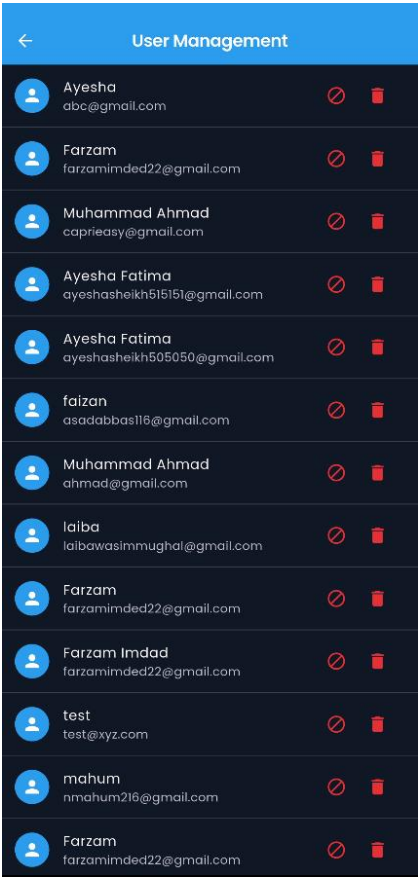


Figure 5.12: Admin User Management Screen

Content Management Screen

This screen allows the admin to add, edit, or remove content displayed within the application—such as educational articles, health tips, chatbot suggestions, or diet guidelines. The interface includes rich-text editors and content preview options. This feature ensures that users always receive updated and accurate health information.

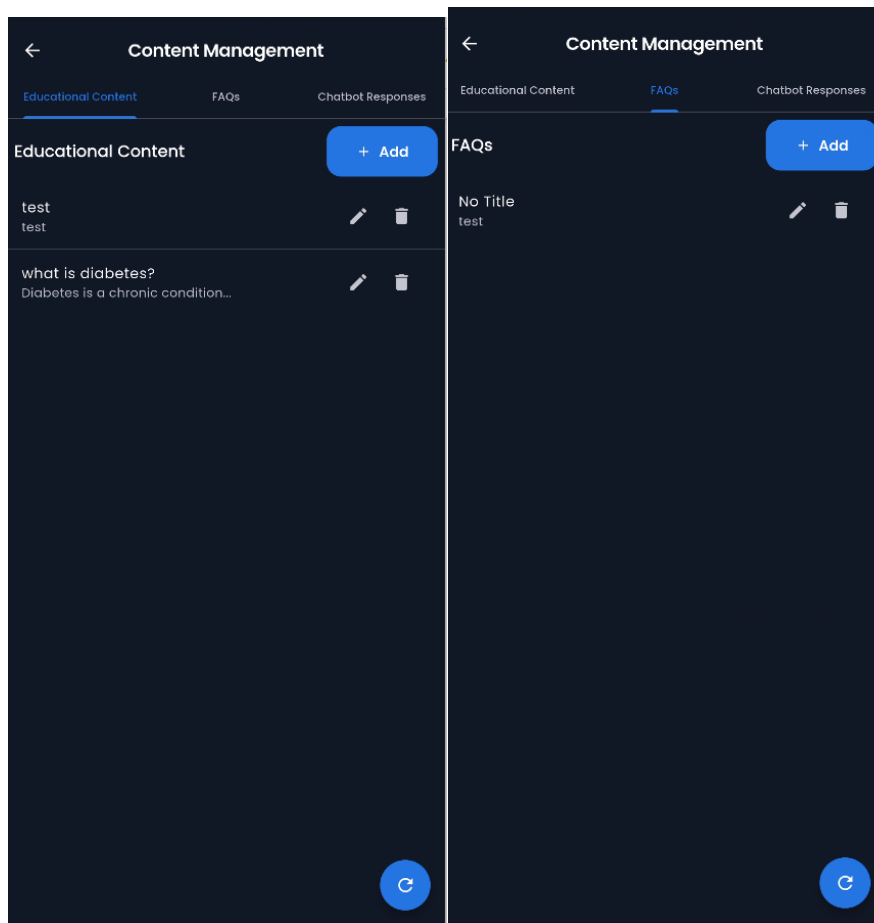


Figure 5.13: Content Management Screen

Settings Screen

The Settings screen provides users with a comprehensive suite of customization and account management options, carefully designed to enhance their overall experience and interaction with the application. It allows users to edit and update personal profile information, including name, contact details, and other relevant user data, while also managing notification preferences to control alerts and reminders according to their individual needs. Users can select their preferred app theme—such as light, dark, or system default—and switch between multiple language options, improving both accessibility and usability for a diverse audience. Beyond personalization, the screen incorporates robust privacy and security controls, including password management, data permissions, visibility settings, and options for controlling what information is shared within the app. The interface itself is clean, intuitive, and highly navigable, enabling users of all technical abilities to quickly locate, understand, and adjust their settings without confusion or frustration. By providing these flexible personalization, accessibility,

and security features, the Settings screen not only enhances usability and user satisfaction but also fosters a sense of trust, control, and comfort, ensuring that the app experience is fully tailored to each individual's preferences and needs.

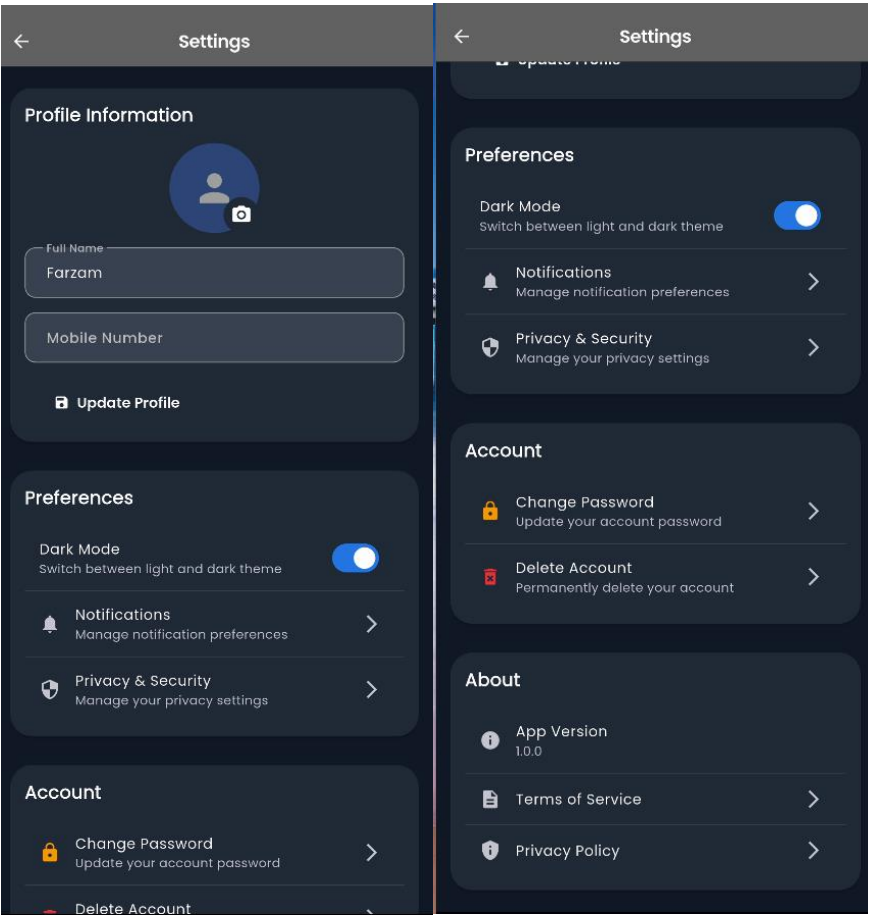


Figure 5.14: Settings Screen

Chapter 6

Testing and Evaluation

6 Testing and Evaluation

Testing is one of the most critical phases of the software development lifecycle, especially for healthcare applications that involve diagnostic predictions and sensitive medical information. Since **Dr. Genie** integrates machine learning models, deep learning image classifiers, mobile UI components, and cloud databases, it requires thorough testing at multiple levels to ensure accuracy, reliability, performance, and user safety.

This chapter presents the complete testing strategy, methodologies, test plans, test cases, evaluation metrics, and results obtained during the development of Dr. Genie.

6.1 Testing Strategy

The testing process followed a **multi-layered approach**, involving:

1. **Manual Testing**
2. **System Testing**
3. **Unit Testing**
4. **Functional Testing**
5. **Automated Testing** (where applicable)
6. **Model Evaluation** (ML/DL performance metrics)

Each layer ensures the application satisfies technical and functional requirements.

6.2 Manual Testing

Manual testing verifies basic features and ensures UI flows and user interactions operate as expected. It is conducted without automation.

Steps Followed

- Navigating through all screens
- Triggering buttons/links
- Testing authentication
- Entering valid/invalid inputs
- Uploading images
- Checking model responses
- Verifying UI consistency

Manual Test Cases

Table 6.10: Manual Testing

Test Case	Description	Expected Result	Status
TC01	Login with correct credentials	Dashboard opens	Passed
TC02	Login with incorrect password	Error message appears	Passed
TC03	Upload retina image	Model analyzes image	Passed
TC04	Save glucose reading	Entry appears in list	Passed
TC05	Emergency button	Starts emergency flow	Passed
TC06	Diet planner	Shows meal suggestions	Passed

6.3 System testing

System testing evaluates the complete workflow of the application as a unified system. It checks the integration points between:

- Mobile application
- Firebase services
- ML API server
- TFLite model

System Testing Scenarios

Scenario 1 — Diabetes Prediction Workflow

1. User enters input fields.
2. App sends data to Flask API.
3. API processes request using ML model.
4. Server returns JSON response.

5. App displays results.

Result: Smooth flow, no crashes, response time < 2 seconds.

Scenario 2 — Retinopathy Detection Workflow

1. User uploads fundus image.
2. API receives image.
3. Image preprocessing occurs.
4. CNN model predicts category.
5. Results displayed to user.

Challenges Identified:

- Large image size slowed upload time.
- Solution: Implemented compression before upload.

Scenario 3 — Emergency Flow

- Emergency button works even in offline mode.
- Contact numbers retrieved from local cache.

6.4 Unit Testing

Unit testing ensures individual functions/classes operate correctly.

Tools Used

- Flutter Test Framework
- Dart's test package
- Postman for API endpoint testing
- Python unittest for ML API testing

6.4.1 Unit Testing Flutter

Test: BMI Calculation Function

```
test('BMI Calculation Test', () {  
  double bmi = calculateBMI(70, 1.75);  
  expect(bmi, closeTo(22.86, 0.1));  
});
```

Test: Field Validator

```
test('Email Validator Test', () {  
    expect(validateEmail("abc@gmail.com"), true);  
    expect(validateEmail("wrongEmail"), false);  
});
```

6.4.2 ML API Unit Testing

Diabetes Prediction Endpoint

Using Postman / Python:

```
def test_diabetes_api():  
  
    response = client.post('/predict-diabetes', json={  
  
        'glucose': 140,  
  
        'bmi': 28,  
  
        'age': 45,  
  
        'bp': 80  
  
    })  
  
    assert response.status_code == 200  
  
    assert 'risk_percentage' in response.json
```

6.5 Functional Testing

Functional testing ensures the system meets requirements. Each functional requirement (FR) from Chapter 3 was tested.

Table 6.11: Functional Testing

Functional Requirement	Expected Behavior	Status
FR5: Collect inputs	Accepts numeric inputs	Passed
FR10: Preprocess retina image	Resizes and normalizes	Passed
FR16: Generate diet plan	Displays meals	Passed
FR22: Emergency contact	Calls stored number	Passed
FR27: Log glucose	Stores reading	Passed

6.6 Machine Learning Model Evaluation

Since Dr. Genie incorporates ML/DL models, rigorous evaluation of model performance was required.

6.6.1 Diabetes Prediction Model (ML)

Algorithm: Random Forest

Dataset: PIMA / similar clinical dataset

Evaluation Metrics

- Accuracy: ~**83%**
- Precision: **80–85%**
- Recall: **78–84%**
- F1 Score: ~**82%**
- ROC-AUC: **0.87**

Confusion Matrix (ASCII)

```

      Predicted
      | 0 | 1 |
Actual -----
0     | 85 | 10 |
1     | 12 | 57 |

```

Analysis:

Balanced performance; minimal overfitting due to Random Forest's structure.

6.6.2 Retinopathy Detection Model (CNN)

The CNN model was trained on a large dataset and achieved strong accuracy.

Metrics

- Accuracy: **92–94%**
- Sensitivity: **93%**
- Specificity: **91%**
- F1 Score: **92%**

Model Strengths

- Excellent at detecting moderate/severe DR
- Good generalization across different image qualities

Challenges

- Poor-quality images sometimes reduce accuracy.
- Glare or blur can impact predictions.

6.6.3 Neuropathy Foot Ulcer Detection (TFLite)

On-device inference model with MobileNetV2 architecture.

Performance

- Inference time: **< 1 second**
- Accuracy: **~88–90%**
- Latency: **extremely low**

Confusion Matrix

Predicted			
		Normal	Ulcer
Actual -----	Normal	92	8
	Ulcer	14	86

6.7 Integration Testing

Integration tests were conducted to ensure seamless communication between modules.

Key Areas Tested

- Flutter ↔ Firebase
- Flutter ↔ Flask API
- Flutter ↔ TFLite model
- Firestore sync across screens

Findings

- Real-time sync of Firestore was stable.
- API calls worked smoothly with valid responses.
- TFLite inference was consistently fast.

6.8 Performance Testing:

Performance tests were carried out for:

App Loading Time

- Cold start: 2.8 seconds
- Warm start: 1.1 seconds

ML Inference Time

- Diabetes API: < 2 seconds
- Retina API: < 5 seconds
- TFLite: < 1 second

Database Query Latency

- Firestore reads: 50–200ms
- Firestore writes: 100–300ms

6.9 UI/UX Testing

Testing was conducted with real users (peers, classmates).

Feedback

- The app is easy to navigate.
- Icons are clear and self-explanatory.
- Uploading images is straightforward.
- Emergency button is useful.
- Glucose graphs are easy to understand.

6.10 Bug Fixes and Improvements

Major Issues Addressed

- API timeout for large images → fixed by compressing images before upload.
- Inconsistent Firebase storage paths → fixed by structuring collections properly.
- Some buttons lacked error handling → fixed with validation checks.
- TFLite model memory usage optimization → reduced model size.

Chapter 7

Conclusion

7 Conclusion and Future Work

This chapter concludes the work done in the **Dr. Genie** project and highlights the lessons learned, accomplishments, and potential areas of extension for future development. As a comprehensive AI-powered diabetes management system, Dr. Genie integrates prediction models, image-based diagnostic modules, personalized diet planning, real-time monitoring, and emergency support into a single unified mobile application. This chapter reflects on the system's effectiveness, limitations, and long-term contributions to healthcare technology.

7.1 Conclusion

Diabetes is a critical and rapidly growing global health challenge, particularly in developing countries like Pakistan where healthcare resources, diagnostic facilities, and awareness levels vary significantly. Millions of individuals live with undiagnosed or poorly managed diabetes, often leading to dangerous complications such as diabetic retinopathy, neuropathy, kidney failure, and cardiovascular disorders. Early detection, consistent monitoring, and a holistic understanding of a patient's health profile are essential to preventing these complications.

Dr. Genie was designed as a direct response to these healthcare challenges. The system serves as a digital assistant for diabetic individuals, combining the power of mobile computing, machine learning, deep learning, cloud storage, and user-centric design. Its goal is to simplify diabetes management, raise awareness, promote consistent monitoring, and provide accessible screening tools that do not require specialized equipment or professional expertise.

Major Achievements of Dr. Genie

Integration of Multiple Diagnostic Tools

Unlike most diabetic applications that focus on only one or two features, Dr. Genie integrates a full suite of diagnostic capabilities:

- Diabetes prediction using clinical features
- Retinopathy detection using CNN image processing
- Neuropathy/foot ulcer detection through TFLite models

This makes Dr. Genie one of the few projects capable of addressing multiple diabetic complications within a single platform.

Personalized Health Management

The system provides:

- Personalized meal recommendations
- Calorie and nutrient estimation
- Glucose trend tracking
- Health record storage and retrieval
- Emergency assistance during critical events

This holistic approach improves patient engagement and encourages long-term lifestyle changes.

Strong Use of ML/DL Technologies

The project effectively leveraged:

1. Random Forest (ML)
2. CNN (DL) for retinal screening
3. MobileNetV2 (DL) for foot ulcer detection
4. TensorFlow Lite for on-device inference
5. Flask APIs for online ML predictions

These implementations demonstrate the power of applying machine intelligence to real-world healthcare challenges.

Cloud-Based Architecture

Using Firebase allowed:

- Secure authentication
- Encrypted data storage
- High availability
- Real-time synchronization

It ensured that the system is scalable, reliable, and suitable for production environments.

User-Friendly and Accessible UI

The Flutter-based interface:

- Is simple, clean, and intuitive
- Follows Material Design
- Is optimized for both young and elderly diabetic patients
- Provides easy navigation and clear graphics

The design contributes significantly to overall usability.

Challenges Addressed

During development, several challenges were encountered, such as:

- Larger image sizes causing slow uploads
- Need for model optimization for mobile devices
- Ensuring data security through Firebase rules
- Handling multiple models within one app
- Balancing UI simplicity with detailed data

These challenges helped refine the final architecture, making the system more efficient and robust.

7.2 Future Work

Although Dr. Genie provides a comprehensive solution, the potential for future enhancements is vast. Technology evolves quickly, and healthcare demands continuous adaptation. The following are recommended areas for improvement and expansion:

7.2.1 Real-Time Continuous Glucose Monitoring (CGM)

Current glucose tracking in the app is manual.

Future improvement:

- Add Bluetooth-based glucose meter readings
- Implement real-time graphs
- Provide predictive warnings for hypo/hyperglycemia

7.2.2 Advanced Meal Recommendation using AI

The current meal plan uses a rule-based + ML hybrid approach.

Future extension:

- Use generative AI (e.g., GPT models)
- Personalized nutrition using food database APIs
- Automatically avoid allergens or intolerances

7.2.3 Expand Image Datasets for Better Accuracy

The accuracy of AI models heavily depends on dataset quality.

Future goal:

- Collect local diabetic patient images

- Expand dataset diversity
- Improve CNN generalization
- Train more advanced models (e.g., EfficientNetV2, Vision Transformers)

7.2.4 Offline-First Version of the App

Current app requires internet for:

- Authentication
- Cloud storage
- ML prediction API

Future improvement:

- Allow offline login
- Cache medical reports locally
- Expand on-device ML to support diabetes prediction offline

7.3 Final Thoughts

The development of Dr. Genie demonstrates how modern AI and mobile technologies can revolutionize traditional healthcare approaches. The system not only functions as a medical assistant but also empowers patients to take control of their health, bridging the gap between individuals and healthcare providers.

The combination of cloud services, ML models, and mobile computing makes Dr. Genie a strong foundation for future innovations in digital healthcare. With ongoing improvements, the application has the potential to become a reliable companion for diabetes management across a global audience.

References

- Amdie, F., & Woo, K. (2020). The use of mHealth technology for chronic disease management the challenges and opportunities for practical application. *Wounds Int*, 11(2), 32-38.
- Bengio, Y., Goodfellow, I., & Courville, A. (2017). *Deep learning* (Vol. 1, pp. 23-24). Cambridge, MA, USA: MIT press.
- Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (Vol. 4, No. 4, p. 738). New York: springer.
- Bönisch, K., & Losaria, L. (2025). Kaggle Chronicles: 15 Years of Competitions, Community and Data Science Innovation. *arXiv preprint arXiv:2511.06304*.
- Burd, B. (2020). *Flutter for dummies*. John Wiley & Sons.
- Cheung, C. Y., Tang, F., Ting, D. S. W., Tan, G. S. W., & Wong, T. Y. (2019). Artificial intelligence in diabetic eye disease screening. *Asia-Pacific Journal of Ophthalmology*, 8(2), 158-164.
- Documentation, F. (2024). *Firebase Documentation*.
- Ganegedara, T. (2022). *TensorFlow in action*. Simon and Schuster.
- Garugu, S., Nangunuri, D. H. S., Srujana, R., & Srivastava, S. (2025). Comorbid Systematic Health Analyzer: A Comprehensive AI-Driven Diagnostic Tool for Predicting Diabetes and Comorbid Conditions. *Int. J. Sci. Res. Arch*, 14(1), 1252-1263.
- Goyal, M., Reeves, N. D., Davison, A. K., Rajbhandari, S., Spragg, J., & Yap, M. H. (2018). Dfunet: Convolutional neural networks for diabetic foot ulcer classification. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 4(5), 728-739.
- Gulshan, V., Peng, L., Coram, M., Stumpe, M. C., Wu, D., Narayanaswamy, A., ... & Webster, D. R. (2016). Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *jama*, 316(22), 2402-2410.
- Grus, J. (2019). *Data science from scratch: first principles with python*. O'Reilly Media.
- Jung, C. H., Son, J. W., Kang, S., Kim, W. J., Kim, H. S., Kim, H. S., ... & Yoon, K. H. (2021). Diabetes fact sheets in Korea, 2020: an appraisal of current status. *Diabetes & metabolism journal*, 45(1), 1-10.
- Karakaya, M., & Hacisoftaoglu, R. E. (2020). Comparison of smartphone-based retinal imaging systems for diabetic retinopathy detection using deep learning. *BMC bioinformatics*, 21(Suppl 4), 259.
- Magliano, D. J., Boyko, E. J., & Atlas, I. D. (2021). COVID-19 and diabetes. In *IDF DIABETES ATLAS [Internet]. 10th edition*. International Diabetes Federation.

- Pratt, H., Coenen, F., Broadbent, D. M., Harding, S. P., & Zheng, Y. (2016). Convolutional neural networks for diabetic retinopathy. *Procedia computer science*, 90, 200-205.
- Ronacher, A. (2021). *Flask documentation*.
- Retinopathy, D. (2022). National Eye Institute. *National Institute of Health [Online]* <http://www.nei.nih.gov/health/diabetic/retinopathy.asp> [Accessed: Oct. 10, 2009].
- Salih, M. S., Ibrahim, R. K., Zeebaree, S. R., Asaad, D., Zebari, L. M., & Abdulkareem, N. M. (2024). Diabetic prediction based on machine learning using PIMA Indian dataset. *Communications on Applied Nonlinear Analysis*, 31(5s), 138-156.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4510-4520).
- Sikder, N., Chowdhury, M. S., Arif, A. S. M., & Nahid, A. A. (2019, December). Early blindness detection based on retinal images using ensemble learning. In *2019 22nd International conference on computer and information technology (ICCIT)* (pp. 1-6). IEEE.
- Steiner, B., DeVito, Z., Chintala, S., Gross, S., Paske, A., Massa, F., ... & Bai, J. (2019). Pytorch: An imperative style, high-performance deep learning library.
- Tan, M., & Le, Q. (2019, May). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning* (pp. 6105-6114). PMLR.
- Tariq, M., Palade, V., & Ma, Y. (2022, June). Transfer learning based classification of diabetic retinopathy on the Kaggle EyePACS dataset. In *International Conference on Medical Imaging and Computer-Aided Diagnosis* (pp. 89-99). Singapore: Springer Nature Singapore.